Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

École Polytechnique Internationale Privée de Tunis

Projet de Fin d'Études

Pour obtenir le diplôme de

Mastère Professionnel

Spécialité : Génie Logiciel

Présenté et réalisé par :

Développement d'une plateforme intelligente pour la gestion des partenariats d'entreprise

Mohamed Achref Ben Abdallah

Juillet 2026

Préparé au sein de :

Capgemini Engineering Tunisie

JURY

Mme. _____	Enseignante à Polytech Intl	Présidente
M. _____	_____	Rapporteur
Mme. Eya Khemiri	Capgemini Engineering Tunisie	Encadrant Professionnel
Dr. Aymen Louati	Enseignant à Polytech Intl	Encadrant Académique

FOR.29 | V.01

Année Universitaire : **2025/2026**

Abstract

IntelliConnect is an enterprise partnership and project management platform designed and built for Capgemini Engineering Tunisie. The platform gives the company a single, secure workspace to manage its network of partners and its portfolio of projects, replacing scattered files and disconnected tools with one governed system. It brings partner and project data together, tracks activity and commitments over time, and helps teams schedule and follow the work that keeps each relationship active. Role-based access control and a unified sign-in give every internal role, every external partner, and every public applicant exactly the view and the permissions their responsibilities require. A companion human-resources platform is built as a sister project, and IntelliConnect is designed to connect to it so that talent and recruitment data flow into the same partnership picture.

Beyond day-to-day operations, IntelliConnect turns partnership data into decision support. It combines advanced scoring and ranking techniques with vector-based semantic matching to produce clear health scores for both partners and projects, so analysts can see where attention is needed before a situation deteriorates. The platform also applies current agentic AI and harness-engineering practices to deliver a production-grade assistant that helps analysts produce executive reports which meet the company's quality standards and can be exported to PDF or Excel. The whole system is built for production use, with sustained attention to reliability, observability, and traceable AI behaviour, so that the intelligence it provides can be trusted in a real enterprise setting.

Keywords: enterprise partnership management, project management, role-based access control, partner and project health scoring, semantic matching, agentic AI, report generation, production observability.

Résumé

IntelliConnect est une plateforme de gestion des partenariats et des projets d'entreprise conçue et développée pour Capgemini Engineering Tunisie. La plateforme offre à l'entreprise un espace de travail unique et sécurisé pour piloter son réseau de partenaires et son portefeuille de projets, en remplaçant des fichiers dispersés et des outils déconnectés par un seul système gouverné. Elle rassemble les données des partenaires et des projets, suit l'activité et les engagements dans le temps, et aide les équipes à planifier et à suivre le travail qui maintient chaque relation active. Le contrôle d'accès basé sur les rôles et une connexion unifiée donnent à chaque rôle interne, à chaque partenaire externe et à chaque candidat public exactement la vue et les droits que ses responsabilités exigent. Une plateforme de gestion des ressources humaines est développée comme projet jumeau, et IntelliConnect est conçue pour s'y connecter afin que les données de talents et de recrutement alimentent la même vision partenariale.

Au-delà des opérations quotidiennes, IntelliConnect transforme les données partenariales en aide à la décision. Elle associe des techniques avancées de scoring et de classement à un appariement sémantique vectoriel pour produire des scores de santé clairs, aussi bien pour les partenaires que pour les projets, afin que les analystes puissent repérer les points d'attention avant qu'une situation ne se dégrade. La plateforme applique également les pratiques actuelles d'IA agentique et d'ingénierie de harnais pour offrir un assistant de niveau production qui aide les analystes à produire des rapports exécutifs conformes aux standards de qualité de l'entreprise et exportables en PDF ou en Excel. L'ensemble du système est bâti pour un usage en production, avec un soin constant porté à la fiabilité, à l'observabilité et à la traçabilité du comportement de l'IA, afin que l'intelligence fournie soit digne de confiance dans un contexte d'entreprise réel.

Mots-clés : gestion des partenariats d'entreprise, gestion de projets, contrôle d'accès basé sur les rôles, scoring de santé des partenaires et des projets, appariement sémantique, IA agentique, génération de rapports, observabilité en production.

Contents

List of Figures

List of Tables

Mots-clés et acronymes

Mots-clés

Français : gestion des partenariats, CRM, IA agentique, agent conversationnel, *tool calling*, RAG, pgvector, scoring de partenaires, Next.js, TypeScript, Drizzle ORM, observabilité LLM.

English: partnership management, CRM, agentic AI, conversational agent, tool calling, RAG, pgvector, partner scoring, Next.js, TypeScript, Drizzle ORM, LLM observability.

Liste des acronymes

Acronyme	Signification
AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
BI	Business Intelligence
CDI	Contrat à Durée Indéterminée
CRM	Customer Relationship Management
DW	Data Warehouse
EUR-ACE	European Accreditation of Engineering Programmes
HR / RH	Human Resources / Ressources Humaines
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
ORM	Object-Relational Mapping
PDF	Portable Document Format
PRM	Partner Relationship Management
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
SSE	Server-Sent Events
UML	Unified Modeling Language

General Introduction

Enterprise competitiveness increasingly depends on how well an organization manages the relationships around it. A company such as Capgemini Engineering Tunisie works with corporate customers, technology and service suppliers, marketing organizations, and engineering schools. Each of these relationships carries its own contacts, commercial terms, shared projects, documents, and obligations. When this network grows, managing it through email threads, spreadsheets, and shared drives becomes error-prone. Renewals slip, warning signs stay hidden until attrition is already under way, and teams lose the common view they need to decide where to focus.

IntelliConnect was designed and built during a master's final-year internship to replace that fragmented tooling with one governed platform. It centralizes the full partnership lifecycle and the project portfolio behind a secure, role-aware interface, and it adds an intelligence layer that turns operational data into grounded decision support.

The platform is organized around three surfaces. The public website presents the company's partnership offer and lets an organization submit a partnership request that enters the internal review queue directly. The employee portal is the operational hub for Capgemini staff, giving access to partners, communications, documents, project delivery, business-intelligence dashboards, and the AI assistant. The partner portal gives each external organization a secure self-service space adapted to its category, covering customers, suppliers, marketing partners, and universities.

The intelligence layer is the distinguishing contribution of the project. IntelliConnect implements an agentic assistant built on a streaming conversational interface and a typed, validated tool set, where every answer is grounded in real data from the live database. It combines deterministic partner and project scoring with semantic document retrieval, and it produces executive reports that analysts can export for real use.

These choices are driven by a single question that runs through the report:

> *How can a unified, role-aware platform cover the full operational lifecycle of enterprise partnerships and projects while embedding a governed agentic AI layer whose outputs are grounded, measurable, and trustworthy in a production context?*

The remainder of this report is organized into seven chapters.

- **Chapter 1, General Presentation and Methodology**, introduces the host company, the business context of enterprise partnership management, the project objec-

tives, the positioning of IntelliConnect against existing solutions, and the agile method followed during the internship.

- **Chapter 2, Planning and Architecture**, defines the functional and non-functional requirements, the product backlog and sprint plan, the use-case and data models, the overall application architecture, and the technical environment.
- **Chapter 3, Access, Identity and Auditability**, covers authentication, the role-based access-control model, session handling, and the audit trail that records every sensitive operation across the platform.
- **Chapter 4, Partnership Lifecycle Management and Customers**, presents the partner registry and status workflow, the contact, offer, and event modules, secure document handling, notifications, and the public request intake and review pipeline.
- **Chapter 5, Project Delivery, Operations and Business Intelligence**, covers project delivery, the human-resources workflows, and the data-warehouse-backed analytics dashboard.
- **Chapter 6, Agentic AI Harness and Generative Interface**, examines the conversational agent in depth: its validated tool set, the streaming protocol, multi-step plan execution and the live plan interface, in-line charts and tables, and the report-generation pipeline.
- **Chapter 7, Knowledge Retrieval, Scoring and Observability**, covers document retrieval, the partner and project scoring models, churn-risk prediction, recommendations, and the evaluation and observability framework used to keep AI output measurable.

A **General Conclusion** closes the report with a synthesis of the achieved results, an assessment against the initial objectives, and perspectives for future development and industrialization.

Chapter 1

General Presentation and Methodology

Introduction

This chapter sets the organizational, business, and methodological context of the final year project. Carried out within Capgemini Engineering Tunisie during a full-time internship, Development of an Intelligent Platform for Enterprise Partnership Management produced IntelliConnect, an intelligent web platform for enterprise partnership and project management. It presents Capgemini and Capgemini Engineering Tunisie, defines the business problem, outlines the project objectives and functional surfaces, reviews the state of the art to position the platform, and justifies the Agile Scrum choice and sprint cadence.

1.1 Host Company Presentation

1.1.1 Capgemini Group

Capgemini is a global business and technology transformation group covering consulting, technology services, engineering, data, artificial intelligence, cloud, and digital operations. It operates internationally as a partner for companies that want to transform their processes, systems, and digital products through technology and engineering expertise [1]. This profile is directly relevant to the project because the group works with customers, technology providers, universities, and institutional actors, relies on structured collaboration between internal teams and external partners, and has an engineering culture suited to a platform combining workflows, analytics, and artificial intelligence.

1.1.2 Capgemini Engineering Tunisie

Capgemini Engineering is the engineering and research and development branch of Capgemini. It focuses on intelligent industry, product engineering, software engineering, digital continuity, systems engineering, and applied technology services [2]. In Tunisia, Capgemini Engineering Tunisie extends this positioning within a local ecosystem of

customers, suppliers, academic institutions, and business partners. This local context gives the project a concrete business basis: partnership management covers commercial opportunities, supplier relationships, university cooperation, marketing actions, events, projects, documents, contacts, performance indicators, and governance follow-up. A platform built for this environment must be reliable, role-aware, traceable, and adaptable to different partner categories.

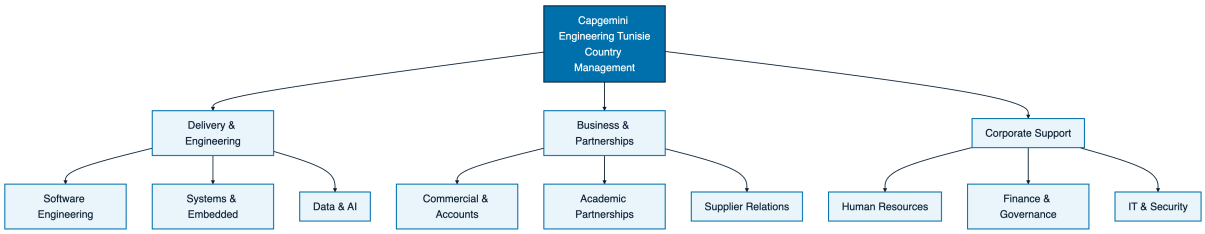


Figure 1.1: Organizational hierarchy and departments of Capgemini Engineering Tunisie.

1.1.3 Relevant Partnership Stakeholders

The useful organizational scope for this project is the set of stakeholders who interact with the partnership lifecycle and need reliable information to make decisions. The most relevant stakeholders are summarized below.

Table 1.1: Relevant stakeholders for partnership management

Stakeholder	Role in the partnership lifecycle
Executive and management teams	Monitor strategic value, portfolio health, performance indicators, and major partnership decisions.
Commercial teams	Follow customer and market-facing relationships, offers, contacts, meetings, opportunities, and negotiation status.
Project and delivery teams	Connect supplier and customer relationships to concrete projects, milestones, workload, risks, and delivery indicators.
Human resources teams	Coordinate university cooperation, student programs, employee information, and internal collaboration needs linked to partners.
Partner representatives	Access their dedicated portal to follow shared information such as profile data, events, documents, offers, notifications, and category-specific activities.
Platform administrators	Govern accounts, permissions, portfolio consistency, activity logs, reporting, and operational supervision.

These stakeholders explain the need for several surfaces rather than a single back-office screen: public visitors need a clear entry point, employees need a complete operational workspace, and external partners need a secure self-service portal.

1.2 Project Presentation

1.2.1 Problem Statement

Enterprise partnership management is often fragmented across spreadsheets, email threads, isolated documents, informal follow-ups, and unrelated tools. As a result, partner information is scattered, meetings, events, offers, projects, and documents are disconnected, relationship status is unclear, and managers lack consolidated indicators when prioritizing action. In the context of Capgemini Engineering Tunisie, partnership relationships cover customers, marketing partners, suppliers, and universities. Each category has specific expectations, but all require structured data, traceability, communication history, performance indicators, and controlled access. The challenge is to manage the full partnership lifecycle through a unified, intelligent, role-based platform. The project addresses the following central question: how can Capgemini Engineering Tunisie centralize and govern its enterprise partnerships while giving operational teams, decision-makers, and external partners useful digital services and AI-assisted insights?

1.2.2 Project Objectives

The objectives of IntelliConnect cover business, functional, technical, and methodological expectations. They are summarized in the following table.

Table 1.2: Main objectives of IntelliConnect

Objective axis	Expected result
Centralization	Provide a unified source of information for partners, contacts, documents, events, offers, projects, requests, and relationship history.
Lifecycle governance	Support partner creation, categorization, status evolution, follow-up actions, notifications, and traceability.
Multi-surface access	Offer a public website, a protected employee portal, and a protected partner portal, each adapted to its audience.
Decision support	Provide portfolio metrics, scoring, dashboards, reports, and AI-assisted recommendations to help teams prioritize actions.
Collaboration	Improve communication between internal teams and partners through shared data, documents, events, notifications, and category-specific services.
Demonstrability	Deliver a coherent final year project that can be presented, tested, and evaluated through realistic scenarios and seeded data.
Maintainability	Build a structured application that separates routing, interface components, server services, database access, authentication, and AI capabilities.

1.2.3 Proposed Solution

The proposed solution is IntelliConnect, a full-stack intelligent web platform for enterprise partnership management. It is organized around three complementary surfaces, each serving a distinct audience with its own dashboard and feature set, as shown in Figure 1.2.

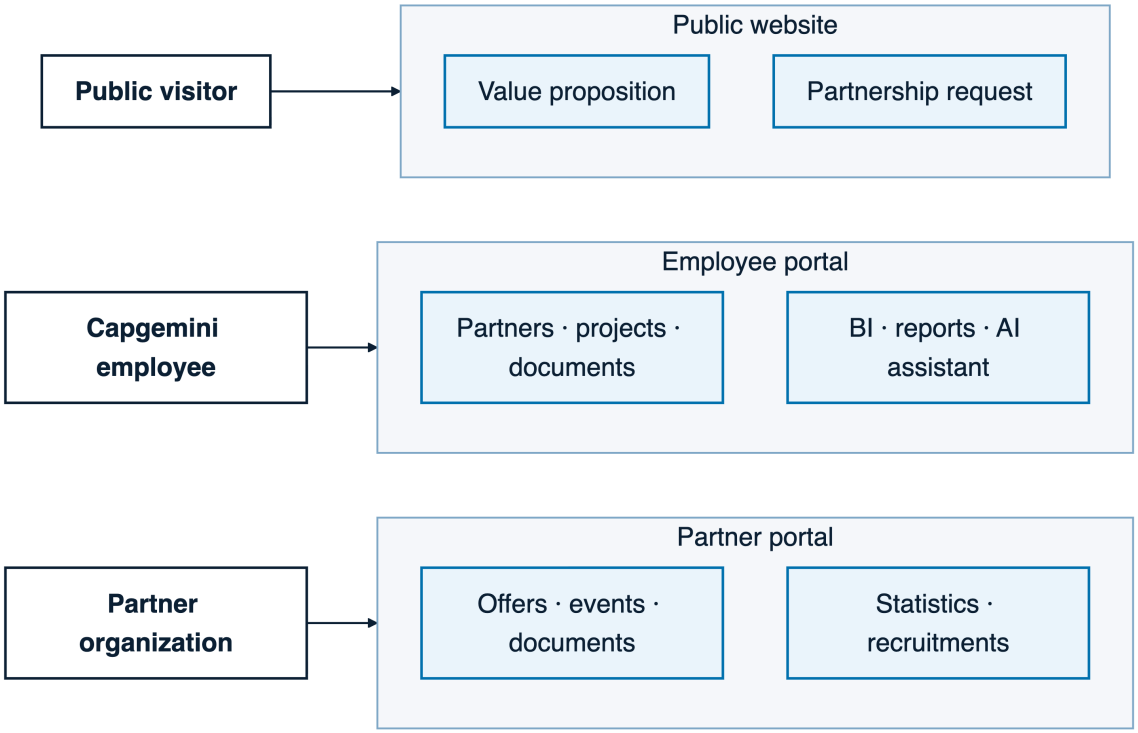


Figure 1.2: The three product surfaces of IntelliConnect and their audiences.

Table 1.3: Main product surfaces of IntelliConnect

Surface	Target users	Main purpose
Public surface	Visitors and potential partners	Present Capgemini partnership value, show collaboration models, display success stories, and collect partnership requests through a public form.
Employee surface	Internal Capgemini users	Manage the partner portfolio, requests, contacts, events, offers, documents, projects, HR cooperation workflows, BI dashboards, reports, and the AI assistant.
Partner surface	External partner representatives	Give partners a secure self-service space to consult profile data, offers, events, documents, notifications, statistics, contacts, and category-specific operations.

The solution also includes an AI-assisted layer. Its role is to summarize information, search partner documents, calculate scores, identify risks, generate analytical reports, and help employees explore portfolio data. The platform combines operational management

and intelligent assistance while keeping the business data structured and governed.

1.3 State of the Art and Positioning

This section positions IntelliConnect against existing tooling on two levels: the market platforms that address relationship management, and the technical foundations of its intelligence layer.

1.3.1 Relationship and Partner Management Platforms

Customer Relationship Management (CRM) platforms such as HubSpot organise contacts, sales pipelines, and customer interactions [5]. Partner Relationship Management (PRM) modules, including the Salesforce partner capabilities, add channel collaboration and shared workflows [4]. These products are mature and broad, but general-purpose: covering the exact scope expected here, namely four first-class partner categories, native public request intake, an integrated partner portal, project delivery, and a governed AI layer, would require substantial configuration, licensing, and integration. IntelliConnect implements this scope directly for the Capgemini Engineering Tunisie context.

1.3.2 Enterprise AI Assistants

A recent alternative is to add a generic enterprise AI assistant over existing data rather than build a dedicated one. Atlassian Rovo offers search, chat, and agents over a Teamwork Graph, but it targets general knowledge work inside the Atlassian ecosystem and provides no native partner-scoring or partnership-CRM runtime [6]. Salesforce Agentforce is CRM-native and capable, yet it assumes Salesforce as the system of record and carries seat- and credit-based pricing that is heavy for a focused custom product [7]. Microsoft Copilot Studio grounds in Microsoft 365 and Dataverse and can build custom agents, but the domain logic and governance must still be assembled across the Power Platform [8]. A bare retrieval chatbot over company data is lighter still, yet on its own it provides neither deterministic scoring, typed domain actions, nor a role-aware audit trail. Table 1.4 summarises the comparison.

Table 1.4: Positioning of IntelliConnect against enterprise AI assistants

Criterion	Atlassian Rovo	Salesforce Agentforce	MS Copilot Studio	IntelliConnect
Orientation	Atlassian teamwork knowledge work	Salesforce CRM automation	Microsoft 365 productivity	Enterprise partnership and project management
Partnership-CRM fit	Generic	Native CRM, not partnership-specific	Generic, assembled	Purpose-built
Deterministic explainable scoring	No	Custom build required	Custom build required	Native five-dimension model
Typed domain tools	General agents	Platform-dependent	Platform-dependent	24 typed, validated tools
Data control	Atlassian Cloud	Salesforce Hyperforce	Azure and Microsoft 365	Single-owner self-hosted database
Cost model	Per seat	Seats plus credits	Per seat	Internal build, no per-seat license
MCP interoperability	Yes	Yes	Yes	Roadmap

The comparison motivates the project: a generic assistant answers questions, whereas partnership decisions at Capgemini require reproducible scoring, typed actions bound to real business data, role-aware access, and control over where that data lives. IntelliConnect is therefore built as a domain runtime rather than a chatbot, and interoperability through the Model Context Protocol (MCP) [39], already supported by the assistants above, is kept as an explicit roadmap item.

1.3.3 Technical Foundations

The intelligence layer builds on established techniques. Retrieval-Augmented Generation grounds model answers in retrieved passages rather than parametric memory [27][29]. Dense retrieval with transformer embeddings [30][31] captures semantic proximity that lexical matching misses, while sparse ranking such as BM25 [32] stays strong on exact terms; the two are complementary and can be merged with Reciprocal Rank Fusion [33]. Agentic use of language models, where the model plans and calls typed tools, follows the ReAct pattern and the broader agent literature [34][35]. Partner deduplication and matching draw on classical entity-resolution work [36]. IntelliConnect combines these foundations into the hybrid, agentic design detailed in the chapters dedicated to the agentic assistant and to the retrieval and scoring layer.

1.4 Methodology Adopted

1.4.1 Why Agile Scrum Was Selected

The development of IntelliConnect required an iterative methodology because the product combines public communication, secure access, internal operations, partner self-service, dashboards, project tracking, HR cooperation, document management, and AI-assisted analysis. These dimensions could not be designed and validated as a single block. Agile Scrum was selected because it supports incremental delivery, regular inspection, backlog prioritization, and continuous adaptation. It made it possible to build a usable foundation first, then enrich it through successive increments, with each sprint focused on a coherent capability while priorities could still evolve according to technical findings, functional complexity, and demonstration needs.

1.4.2 Agile and Classical Methodologies Compared

The following table shows why an agile approach suits this project better than a classical sequential one.

Table 1.5: Comparison between agile and classical project management approaches

Criterion	Agile approach	Classical approach
Delivery mode	Incremental delivery through short iterations	Sequential delivery after successive specification, design, implementation, and validation phases
Requirement handling	Requirements can be refined as the product becomes clearer	Requirements are expected to be stable early in the project
Feedback integration	Frequent feedback after each increment	Feedback often arrives late, near the end of the project
Risk control	Technical and functional risks are discovered progressively	Major risks may remain hidden until integration or final validation
Visibility	Regular progress is visible through working features	Progress may be documented but not always visible in a usable product
Fit for IntelliConnect	Strong fit because the platform combines workflows, portals, analytics, and AI capabilities	Weaker fit because late changes would be costly for a multi-surface platform

The project needed a method that could turn a broad vision into demonstrable increments while keeping the final product coherent, which Agile Scrum provides.

1.4.3 Sprint Cadence

The internship spanned roughly seventeen weeks, divided into five implementation sprints after an initial framing effort.

Table 1.6: Sprint cadence across the internship

Period	Sprint focus	Expected increment
Week 1	Framing and backlog preparation	Understand the host context, define the product scope, identify stakeholders, prepare the backlog, and set up the project structure.
Weeks 2 to 4	Sprint 1: Public and access foundation	Build the public entry point, authentication flow, protected navigation structure, and first employee workspace foundation.
Weeks 5 to 7	Sprint 2: Partner portfolio core	Implement partner records, categories, contacts, status management, documents, events, and basic operational follow-up.
Weeks 8 to 10	Sprint 3: Requests and partner portal	Connect public partnership requests to employee review workflows and provide the secured partner self-service portal.
Weeks 11 to 13	Sprint 4: Projects, HR cooperation, and analytics	Add project delivery tracking, human-resources cooperation modules, dashboards, statistics, reporting, and portfolio indicators.
Weeks 14 to 16	Sprint 5: AI assistance and governance	Integrate scoring, recommendations, document search, AI reports, activity supervision, notifications, and governance features.
Week 17	Stabilization and report alignment	Validate the main flows, refine the demonstration data, improve documentation, and align the report chapters with the implemented product.

Conclusion

This chapter presented the project context. It introduced Capgemini and Capgemini Engineering Tunisie, identified the stakeholders concerned by the platform, formulated the business problem, defined the objectives and surfaces of IntelliConnect, and positioned it through a state of the art of relationship platforms, enterprise AI assistants, and the underlying retrieval and agent techniques. It also justified Agile Scrum and outlined the sprint cadence followed during the internship. The next chapter builds on this context by detailing the planning, requirements, and architecture of the solution. It moves from the general presentation of the project to a structured analysis of actors, functional needs, data organization, and technical design.

Chapter 2

Planning and Architecture

Introduction

This chapter turns the product vision of IntelliConnect into the requirements, diagrams, backlog, sprint plan, and architecture used to guide implementation. The platform covers partnership requests, employee operations, partner self-service, business intelligence, document handling, scoring, and AI-assisted decision support.

2.1 Requirements Analysis and Specification

2.1.1 Actor Identification

The platform distinguishes six main actors. Five are internal employee roles with access to the employee portal, while Partner represents external organizations using the self-service portal.

Table 2.1: Actor identification for IntelliConnect

Actor	Scope	Main responsibilities
Admin	Global platform governance	Manages employee accounts, access rules, audit traces, reference data, and consistency.
Manager	Partnership supervision	Reviews partner portfolios, tracks partnerships, validates lifecycle decisions, monitors project health, and uses AI-assisted insights.
Commercial	Business relationship operations	Manages partner contacts, offers, meetings, communications, opportunities, and follow-up for customer, supplier, marketing, and university partnerships.
Analyst	Reporting and intelligence	Explores dashboards, indicators, partner scores, churn signals, document evidence, and warehouse views.
HR	Academic and workforce workflows	Handles roster views, university collaboration workflows, student recruitment records, conversion indicators, and HR partnership events.
Partner	External self-service access	Uses its dashboard, profile, offers, events, contacts, documents, notifications, statistics, and collaboration areas.

2.1.2 Functional Requirements

The functional requirements cover the public surface, employee portal, partner portal, business intelligence, and AI assistant.

Table 2.2: Functional requirements

ID	Area	Requirement
FR1	Access and identity	Authenticate employees and partners, recover the server session, and route each role to its workspace.
FR2	Public partnership entry	Present the partnership value proposition and let external organizations submit structured partnership requests.
FR3	Partner lifecycle	Create, update, categorize, score, negotiate, suspend, and track partners through a controlled lifecycle with status history.
FR4	Relationship communications	Manage contacts, meetings, events, notifications, and communication records linked to partners.
FR5	Documents and offers	Upload, organize, retrieve, and expose partner documents and offers according to ownership and role permissions.
FR6	Partner self-service	Provide external partners with a restricted portal for profile data, offers, events, documents, notifications, contacts, statistics, and collaboration flows.
FR7	Projects and delivery	Track projects, milestones, tasks, team allocation, progress, budget indicators, project documents, and delivery health.
FR8	Business intelligence	Display analytical dashboards backed by a dedicated data warehouse, including portfolio, category, project, and partner-performance metrics.
FR9	Agentic AI assistant	Provide a streamed AI workspace that can call validated tools for partner search, analytics, scoring, recommendations, project health analysis, report generation, and document retrieval.
FR10	Report generation	Generate, preview, edit, and export executive reports built from real platform data and AI-assisted synthesis.
FR11	Governance and audit	Record significant actions, status changes, communication events, AI tool traces, generated reports, and security-relevant access events.
FR12	Configuration and maintainability	Keep reusable UI primitives, server services, database access, AI tools, and authentication utilities separated by responsibility.

2.1.3 Non-Functional Requirements

The non-functional requirements define the quality constraints for IntelliConnect, which handles enterprise relationship data, documents, analytical indicators, and AI-generated recommendations.

Table 2.3: Non-functional requirements

Quality attribute	Requirement	Architectural response
Security	Protected routes, role separation, partner ownership checks, safe session handling, and generic production errors.	Server-side authentication in the access layer, httpOnly cookie-based sessions, role guards, scoped database queries, and structured API errors.
Validation	External inputs must be validated before business execution.	Zod schemas and explicit guards are applied to request bodies, route parameters, query parameters, form payloads, and AI tool inputs.
Performance	Dashboards and AI tools must avoid unnecessary full-table reads and stay responsive under realistic demo data.	Paginated queries, server-side aggregation, selective columns, streaming AI responses, cached analytical shapes where useful, and a dedicated data warehouse for BI workloads.
Auditability	Operational and AI-assisted decisions must remain explainable after execution.	Status history, activity logs, notification records, report records, tool traces, source links, and explicit separation between generated synthesis and persisted facts.
AI reliability	The assistant must not fabricate business results or bypass authorization.	Typed tools, role-aware tool access, database-grounded outputs, retrieval with source evidence, timeouts, step limits, error handling, and evaluation scripts for critical behaviors.
Maintainability	The codebase must remain readable and adaptable during an incremental internship project.	Feature-driven layers, thin route handlers, shared services, reusable UI primitives, strict TypeScript, centralized schema, and clear layer boundaries.

2.2 Analytical Study

2.2.1 Global Use-Case View

The global use-case view shows how the six actors interact with the public website, the employee portal, the partner portal, BI dashboards, and the AI assistant through distinct authorization scopes.

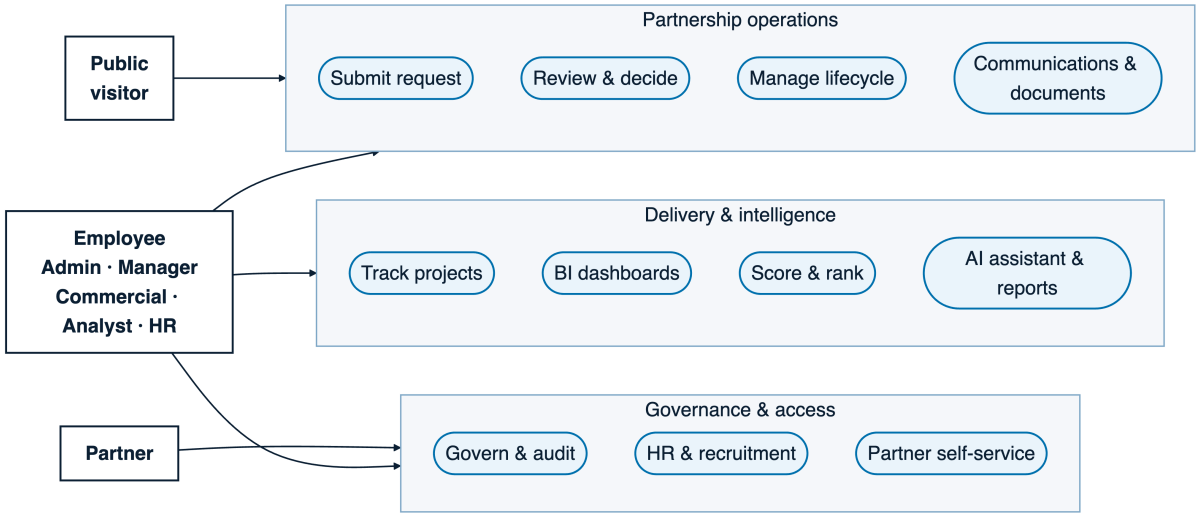


Figure 2.1: Global use-case diagram of IntelliConnect

2.2.2 Class Model View

The class diagram in Figure 2.2 follows the project's UML standard and centers on Partner. A partner belongs to a category and lifecycle status, owns contacts, offers, events, meetings, documents, KPIs, projects, and status history, while employee and partner accounts stay scoped to their own workflows. The AI subsystem uses the same model through grounded tools and document retrieval.

2.2.3 Global Sequence View

Figure 2.3 shows the platform flow: an employee signs in, manages partners, reviews an incoming partnership request, and asks the AI agent for an executive report. The flow spans authenticated access, partnership operations, and grounded report generation.

2.3 Product Backlog and Sprint Planning

2.3.1 Product Backlog

The Product Backlog is organized by epics, so each item maps to a business capability and an architectural slice that can be implemented, verified, and demonstrated progressively.

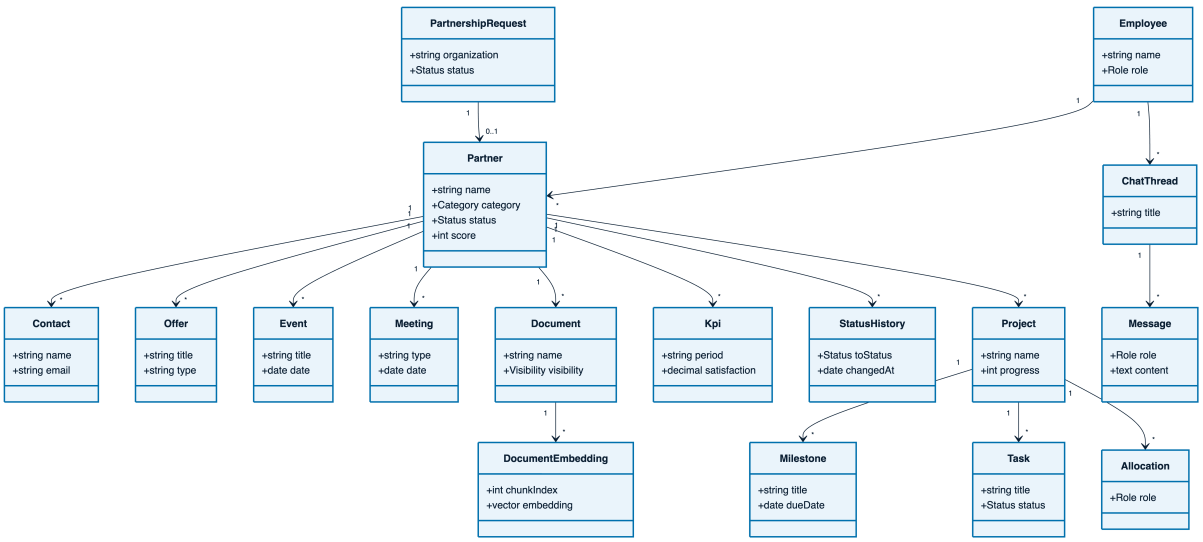


Figure 2.2: Global class diagram of the partnership data model

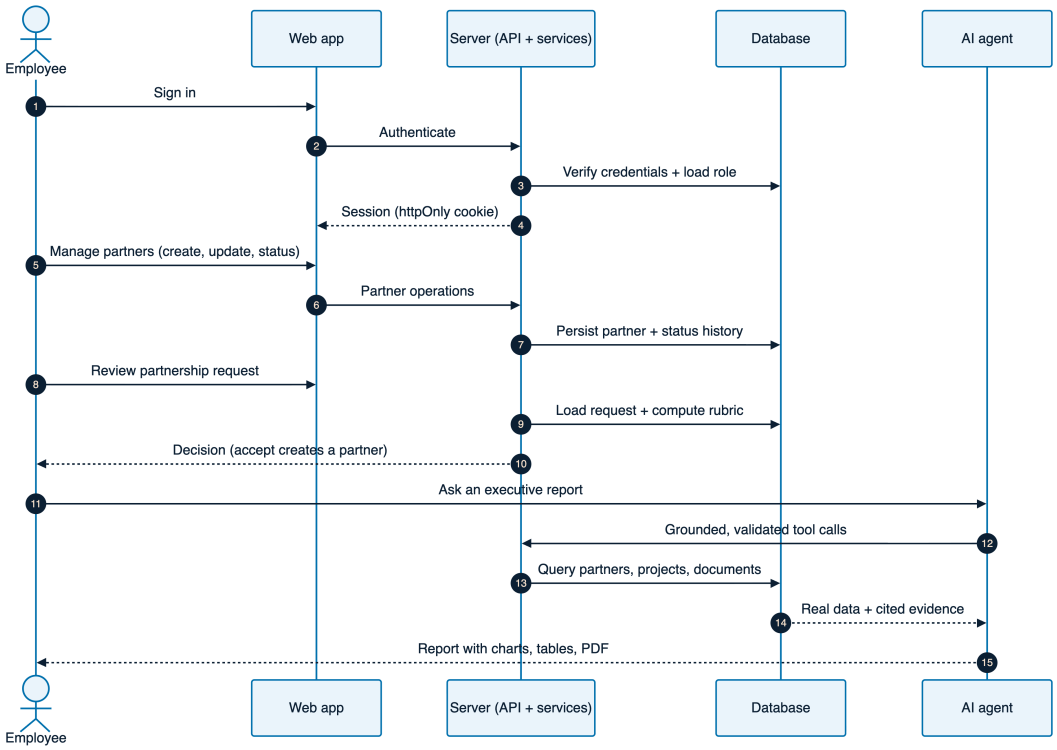


Figure 2.3: Global sequence diagram: from sign-in and partner management to AI report generation

Table 2.4: Product Backlog by epic

ID	Epic	Business value	Main backlog items
PB1	Access and identity	Protect enterprise data and route each user toward the correct workspace.	Public navigation, sign-in, session recovery, role guards, employee shell, partner shell, account governance.
PB2	Partner lifecycle	Maintain a reliable partnership portfolio from request to active collaboration and suspension.	Partnership request review, partner creation, category/status management, scoring page, status history, partner profile editing.
PB3	Relationship communications	Centralize day-to-day relationship activity around each partner.	Contacts, meetings, events, notifications, communication hub, timeline views, partner-specific follow-up.
PB4	Documents and offers	Store and expose operational assets safely for employees and partners.	Document upload and download, ownership checks, offer management, partner-visible documents, report attachments.
PB5	Projects and BI	Connect partnerships with delivery activity and analytical steering.	Projects, milestones, tasks, team allocations, project health, BI charts, data-warehouse refresh, portfolio metrics.
PB6	Agentic AI and RAG	Assist users with grounded analysis and report generation.	Tool-based chat, partner search, scoring tools, recommendations, churn-risk analysis, project health tools, document retrieval, source-grounded reports.
PB7	Governance and audit	Make the platform explainable, reviewable, and maintainable.	Activity logs, status traces, AI tool traces, report records, structured errors, seed scripts, validation scripts, evaluation routines.

This backlog supports incremental delivery. Access and identity come first because later modules depend on authorization, partner lifecycle and CRM communications build the operational base, documents, offers, projects, and BI add collaboration depth, and the AI and RAG epic comes last so responses stay grounded in real data. Governance keeps the platform auditable and safe to evolve.

2.3.2 Sprint Plan

The plan divides the internship into six sprints: foundation, partnership operations, collaboration assets, delivery analytics, AI intelligence, and final hardening.

Table 2.5: Sprint plan across the internship

Sprint	Period	Focus	Expected outcome
Sprint 1	Weeks 1 to 3	Access, identity, and product shell	Public entry, sign-in, protected employee portal, protected partner portal, role-aware navigation, and account governance.
Sprint 2	Weeks 4 to 6	Partner lifecycle	Partner portfolio, request review, category and status management, partner profile views, scoring foundations, and lifecycle history.
Sprint 3	Weeks 7 to 9	Relationship communications and assets	Contacts, meetings, events, notifications, offers, documents, partner self-service flows, and ownership checks.
Sprint 4	Weeks 10 to 12	Projects and business intelligence	Project portfolio, milestones, tasks, team allocation, project health indicators, BI dashboard, and data-warehouse integration.
Sprint 5	Weeks 13 to 15	Agentic AI and retrieval	Streaming AI workspace, typed tools, partner analytics, scoring, recommendations, document retrieval, source evidence, and report generation.
Sprint 6	Weeks 16 to 17	Governance, validation, and final stabilization	Audit traces, error handling, evaluation routines, seed consistency, performance checks, final UI review, and report-ready demonstrations.

The last sprint is shorter and stabilization-oriented so the final project stays demonstrable, documented, and coherent.

2.4 Architecture of the Solution

2.4.1 Architectural Style: Feature-Driven Monolith

IntelliConnect adopts a feature-driven monolith rather than a distributed architecture. The platform shares the same identity model, partner entities, authorization rules, documents, reports, AI traces, and workflows. Splitting those concerns into separate services would add network, deployment, and consistency complexity without a clear benefit at this scale. The monolith stays modular through clear layers: a presentation layer for the

public website and portals, an application layer for authentication and business services, an AI layer for the agent and retrieval, and a data layer for the operational database, the data warehouse, and the vector store.

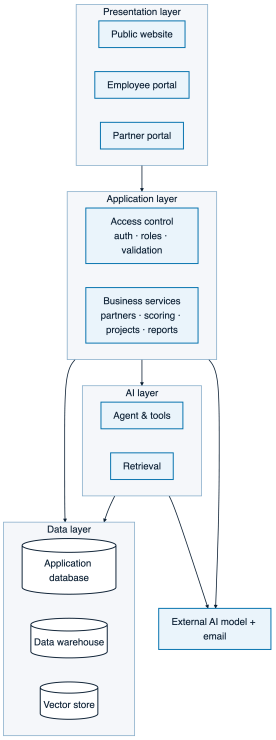


Figure 2.4: Feature-driven monolith architecture of IntelliConnect

The main responsibilities of each layer are:

- **Access layer**: session extraction, password verification, token handling, and role-aware guards for protected requests;
- **Data layer**: the schema, migrations, the operational database client, and the data-warehouse configuration;
- **Service layer**: the business services for partners, scoring, projects, reporting, and shared server operations, which the business-intelligence engineer's module also consumes;
- **AI layer**: agent orchestration, typed tools, the retrieval pipeline, prompts, evaluations, and tool-result formatting.

2.4.2 Layered Organization

The application is a full-stack web app with server-rendered pages, thin request handlers, and a clear split between interface and business logic. Handlers receive input, check the current session, validate the payload, call the appropriate service, and return a structured response. Scoring, partner queries, project health, and report generation stay in the service layer so page code does not become the source of business truth.

2.4.3 External Architecture

From the outside, the system contains these blocks:

- a browser client used by employees, partners, and public visitors;
- the `Next.js` application running the public website, employee portal, partner portal, API routes, and AI chat endpoint;
- the operational PostgreSQL application database storing users, partners, CRM data, projects, documents, reports, conversations, and audit traces;
- the PostgreSQL data-warehouse database storing BI-oriented analytical tables;
- external AI providers used for language generation and optional embeddings;
- an email provider used for platform notifications when configured.

The typical request flow is simple: the browser sends a request to a page or API route, the server authenticates it when required, validated input goes to a service, the service reads or writes through Drizzle ORM [15], PostgreSQL [16] persists the result, and the response returns as HTML, JSON, or a streamed AI message. External providers are used only for specialized capabilities.

2.4.4 AI Subsystem Architecture

The AI subsystem is an authenticated capability embedded in the same platform and governed by the same data-access rules as the rest of the product. Built around the AI SDK [23], typed tools, server-side orchestration, and RAG over curated business documents, it answers business questions, produces analyses, generates reports, searches documents, and calls platform tools without inventing facts.

The subsystem has five layers:

1. **User interaction layer**: the agent workspace, suggestion prompts, conversation history, plan display, tool-result cards, charts, tables, and report cards.
2. **Orchestration layer**: the chat route streams responses, manages tool calls, applies step and timeout limits, and formats assistant output.
3. **Tool layer**: each tool has a typed input schema, a narrow business purpose, and access to server services rather than raw uncontrolled logic.
4. **Retrieval layer**: document chunks and embeddings are stored close to business data; `pgvector` [17] supports semantic retrieval, while metadata and ownership rules keep evidence scoped.
5. **Governance layer**: tool traces, report records, source links, evaluation scripts, and structured failures make AI behavior reviewable.

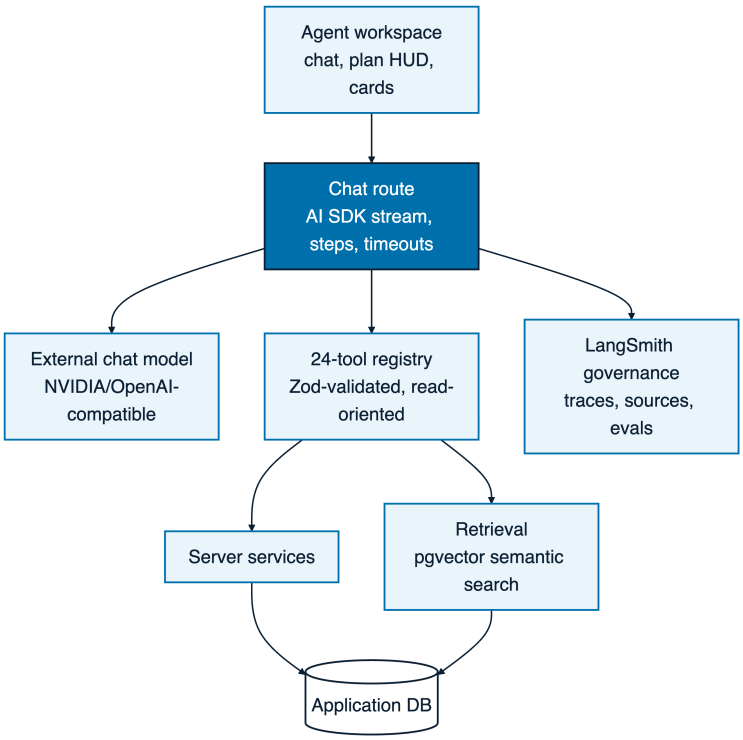


Figure 2.5: Architecture of the agentic AI and RAG subsystem

This design reduces the main risks of AI integration. The assistant stays a controlled interface over validated tools, explains its answers with sources, charts, and tables, and leaves authorization, validation, and persistence to the application.

2.4.5 Two-Database Topology

The platform uses two PostgreSQL databases: an operational application database and a data-warehouse database. This topology separates transactional collaboration data from analytical workloads while keeping the stack understandable and reproducible locally.

Table 2.6: Two-database topology

Database	Main content	Purpose
Application DB	Users, partners, contacts, offers, events, meetings, documents, projects, tasks, notifications, scores, AI threads, reports, audit records, and document chunks.	Supports operational screens, protected workflows, partner self-service, AI tools, RAG evidence, and transactional consistency.
Data warehouse DB	Curated dimensional and fact-oriented tables for portfolio, category, partner, and project indicators.	Supports BI dashboards, aggregated metrics, analytical charts, and read-heavy reporting without overloading operational flows.

The application database is optimized for correctness and workflow execution, while the data warehouse is optimized for analysis and presentation. The separation also keeps demonstrations safer because BI views can evolve without weakening the operational schema.

2.5 Technology Stack

The selected stack supports a modern full-stack web application with strong typing, an efficient developer workflow, a consistent UI system, relational persistence, analytics, and AI integration.

Table 2.7: Technology stack

Area	Technology	Role in the project
Framework	`Next.js` App Router [9]	Full-stack routing, layouts, server components, API routes, streaming endpoints, loading states, and error boundaries.
UI library	`React` [10]	Component-based interface composition for public pages, employee dashboards, partner portal, and AI workspace.
Language	`TypeScript` [11]	Static typing for route contracts, component props, server services, AI tool schemas, and database-facing logic.
Runtime and scripts	`Bun` [12]	Dependency management, local development, build scripts, database scripts, seed routines, and evaluation commands.
Styling	`Tailwind CSS` [13]	Token-driven responsive styling, spacing discipline, light and dark modes, and rapid UI composition.
UI primitives	`shadcn/ui` [14]	Accessible component foundations for buttons, forms, dialogs, tables, alerts, and layout primitives.
ORM	`Drizzle ORM` [15]	Type-safe SQL modeling, schema centralization, migrations, and database queries.
Database	`PostgreSQL` [16]	Operational persistence and analytical storage through the application database and data warehouse.
Vector search	`pgvector` [17]	Storage and similarity search for document embeddings used by RAG.
AI orchestration	`AI SDK` [23]	Streaming chat responses, tool calls, structured outputs, and agentic workflow integration.

Together, these choices produce an architecture that is ambitious enough for an AI-enhanced enterprise platform but still realistic for a final-year project. The same stack supports user-facing pages, server-side business rules, database access, BI, document retrieval, and AI-assisted reporting without multiplying runtime models.

Conclusion

This chapter established the planning and architecture foundation of IntelliConnect. The actor analysis clarified the responsibilities of Admin, Manager, Commercial, Analyst, HR, and Partner users. The functional and non-functional requirements defined what the platform must do and how reliably it must behave. The Product Backlog and six-sprint

plan turned the scope into an incremental roadmap across the internship. Architecturally, the project is a feature-driven monolith with presentation, application, AI, and data layers, built with strict `TypeScript`, reusable components, thin request handlers, and server services. The external architecture keeps the operational core inside the application while using external providers only for specialized AI and email capabilities. The AI subsystem is governed, tool-based, and source-grounded. The two-database topology separates operational persistence from analytical workloads and gives the platform a solid foundation for the implementation chapters that follow.

Chapter 3

Access, Identity and Auditability

Introduction

3.1 Sprint and module objective

3.1.1 Functional objective

The objective of this module is to provide a secure entry point for all IntelliConnect users while keeping public pages, internal employee operations and partner self-service separate.

- provide a public entry surface for visitors without exposing protected data;
- authenticate employees and partners with email and password credentials;
- issue a server-verifiable JWT session stored in an httpOnly cookie;
- extract the session on the server before rendering protected layouts or processing protected API requests;
- route employees and partners toward the correct portal after sign-in;
- enforce role-based permissions for Admin, Manager, Commercial, Analyst, HR and Partner users;
- preserve auditable records for partner status changes and other sensitive actions.

3.1.2 Backlog for access, identity and auditability

Table 3.1: Backlog of the access, identity and auditability module

ID	Theme	User story	Status
AI-1	Public access	As a visitor, I can consult public pages without authentication while protected information remains inaccessible.	Completed
AI-2	Sign-in	As an authorized user, I can sign in with an email and password through a unified access page.	Completed
AI-3	Session security	As the system, I store the authenticated session in an httpOnly cookie and verify it server-side with `jose`.	Completed
AI-4	Password security	As the system, I store only hashed passwords using `bcryptjs` and never persist plain-text credentials.	Completed
AI-5	Portal routing	As an authenticated user, I am redirected to the employee or partner portal according to my account type and role.	Completed
AI-6	RBAC	As an administrator, I can rely on role-based permissions to restrict sensitive screens and operations.	Completed
AI-7	API protection	As the backend, I reject unauthenticated, unauthorized or invalid requests with structured API errors.	Completed
AI-8	Auditability	As a governance user, I can trace partner status changes with actor, previous status, new status, reason and timestamp.	Completed

3.2 Implementation analysis

3.2.1 JWT sessions with httpOnly cookies

3.2.2 Session extraction and server-side trust boundary

3.2.3 Password hashing with bcryptjs

3.2.4 Route protection and role-aware rendering

3.2.5 Structured API errors

```
{ "error": "Validation failed", "details": [...] }
```

The status code reflects the failure category: `401` for missing or invalid authentication, `403`

for insufficient permissions, `400` for invalid input and `500` for unexpected server failures.

3.3 Role-based access control

3.3.1 Access-control flow

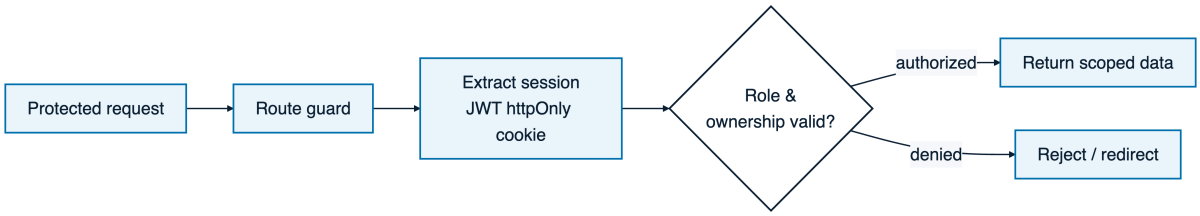


Figure 3.1: Access-control decision applied to every protected request.

3.3.2 RBAC role matrix

Table 3.2: RBAC matrix for IntelliConnect roles

Capability	Admin	Manager	Commercial	Analyst	HR	Partner
Access employee dashboard	Yes	Yes	Yes	Yes	Yes	No
Access partner portal	No	No	No	No	No	Yes
Manage user accounts and roles	Yes	No	No	No	No	No
View partner portfolio	Yes	Yes	Yes	Yes	Yes	Limited
Create and update partner records	Yes	Yes	Yes	No	No	Limited
Review partnership requests	Yes	Yes	Yes	No	No	No
View analytics and BI dashboards	Yes	Yes	Yes	Yes	Yes	Limited
Manage HR workflows	Yes	No	No	No	Yes	No
Upload or manage partner documents	Yes	Yes	Yes	No	No	Own scope
View audit and status history	Yes	Yes	Yes	Yes	Yes	Own scope
Use AI decision-support tools	Yes	Yes	Yes	Yes	Yes	No

3.3.3 Employee and partner portal separation

- The employee portal, exposed under `/dashboard`, centralizes internal operations: partner records, contacts, offers, events, projects, documents, status history, reports, BI and AI-assisted analysis.
- The partner portal, exposed under `/partner`, provides a self-service workspace for external organizations: profile, contacts, notifications, documents, offers, events, statistics and category-specific collaboration features.

3.3.4 Public routes and protected routes

Table 3.3: Public and protected route categories

Category	Examples	Access rule
Public routes	`/`, `/solutions`, `/success-stories`, `/success-stories/apply`, `/privacy`, `/terms`	Available without authentication; they must not expose protected operational data.
Authentication route	`/auth/sign-in`	Available to unauthenticated users; after successful sign-in, the server directs the user to the proper portal.
Employee protected routes	`/dashboard`, `/dashboard/partners`, `/dashboard/bi`, `/dashboard/reports`, `/dashboard/hr/employees`	Require a valid employee session and an authorized employee role.
Partner protected routes	`/partner`, `/partner/documents`, `/partner/profile`, `/partner/stats`	Require a valid partner session and restrict data to the partner organization.
Protected API routes	`/api/...` endpoints serving operational data	Require session extraction, role checks, input validation and structured errors.

3.4 Auditability and traceability

3.4.1 Partner status history

Table 3.4: Information retained for partner status history

Field	Purpose
Actor	Identifies the authenticated user who performed the transition.
Previous status	Preserves the state before the operation, enabling reconstruction of the lifecycle path.
New status	Records the resulting business state after the operation.
Reason	Captures the justification entered by the user or produced by the workflow.
Timestamp	Establishes when the transition occurred for chronological and legal traceability.

3.4.2 Action logs and status transitions

3.4.3 Security and legal traceability

3.5 Implementation

3.5.1 Access use cases

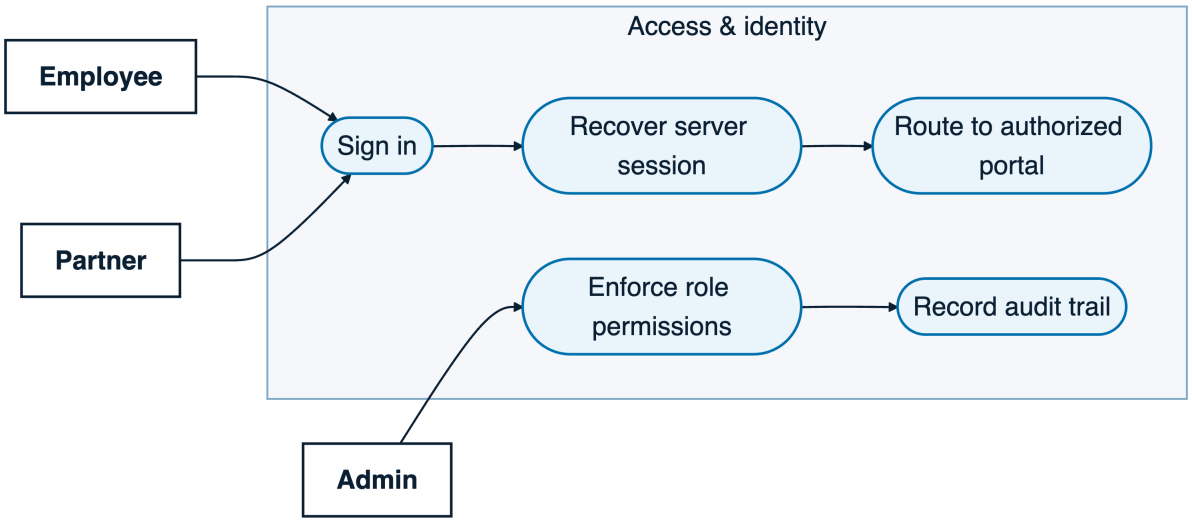


Figure 3.2: Use cases of the access and identity module.

3.5.2 Sign-in page

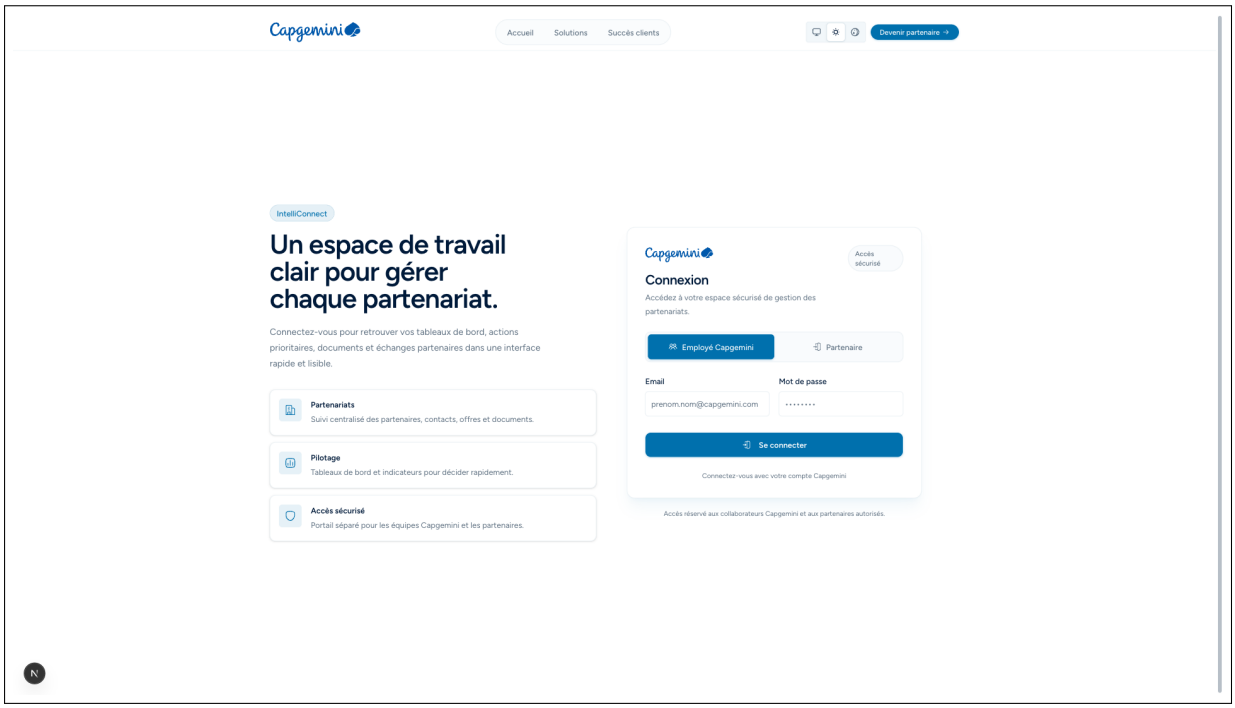


Figure 3.3: Sign-in page used as the controlled entry point to IntelliConnect.

3.5.3 Authentication and role-routing sequence

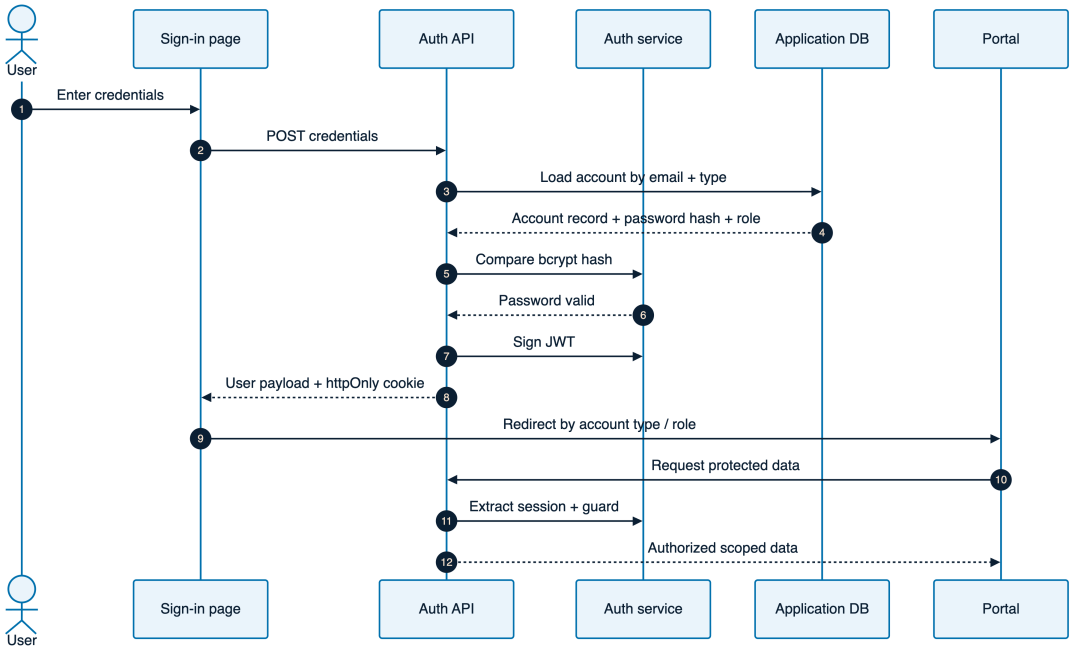


Figure 3.4: JWT creation, session extraction, and role-based portal routing.

Conclusion

This chapter establishes the access, identity and auditability foundation of IntelliConnect. The module combines JWT sessions stored in httpOnly cookies, server-side verification with `jose`, password hashing with `bcryptjs`, protected routing, structured API errors and a role matrix adapted to employee and partner responsibilities.

Chapter 4

Partnership Lifecycle Management and Customers

Introduction

Chapter 4 covers the operational core of IntelliConnect, the module that manages the partnership lifecycle and related relationship-management functions. Once access and roles are set, Capgemini Engineering Tunisie can register partners, assign categories, track status, coordinate exchanges, manage offers and grants, store documents, and review public partnership requests.

It provides an auditable workflow for lifecycle modeling, relationship entities, document governance, public request intake, notifications, and AI-assisted compatibility analysis.

4.1 Sprint 2: module objective and scope

4.1.1 Objective of the module

The objective of this sprint is to implement the partnership and relationship-management layer of IntelliConnect. The module lets an employee:

- create, consult, update, and remove partner records;
- classify each partner into a business category;
- follow a controlled lifecycle from first qualification to active collaboration, suspension, or closure;
- maintain contacts, offers, grants, communications, documents, and notifications for each partner;
- receive public partnership requests, analyze them, and convert validated requests into managed partner records;
- keep an audit trail of status changes and decision history.

4.1.2 Backlog of the module

Table 4.1: Backlog for partnership lifecycle and relationship-management module

ID	Theme	User Story	Status
1	Partner repository	Create, search, filter, edit, and remove partner records to keep a reliable portfolio.	Completed
2	Partner details	Open a partner detail page aggregating identity, category, lifecycle state, contacts, documents, communications, and offers.	Completed
3	Categorization	Classify partners as customer, marketing, supplier/technology, or university partners.	Completed
4	Lifecycle control	Update partner status and preserve status history so operational decisions remain traceable.	Completed
5	Contacts directory	Manage named contacts attached to partners and consult a global contacts directory.	Completed
6	Offers and grants	Manage collaboration offers and grants with clear ownership and status.	Completed
7	Communications	Plan scheduled events and meetings linked to partners.	Completed
8	Documents	Upload, download, preview, and own partner documents.	Completed
9	Notifications	Notify users about relevant partner activity and pending actions.	Completed
10	Public requests	Submit a partnership or adherence request from the public website.	Completed
11	Review queue	Review incoming requests, consult AI-assisted compatibility analysis, and accept or reject them.	Completed

4.2 Partner categories

IntelliConnect models the partner portfolio through four categories. They share one relationship foundation, but their business meaning, expected data, and downstream use differ (Figure 4.1).

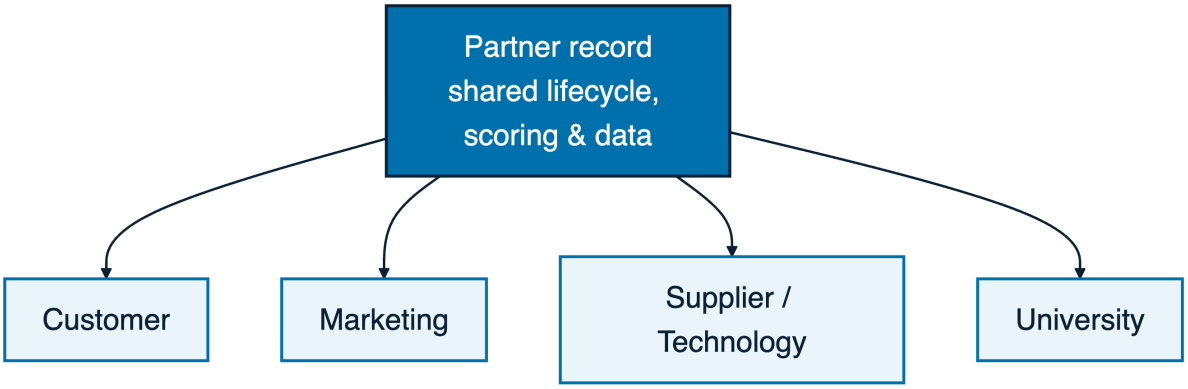


Figure 4.1: Partner categories over a shared partner record.

Table 4.2: Partner categories supported by the relationship management module

Category	Business role	Typical relationship management focus
Customer	Organization that collaborates with Capgemini Engineering Tunisie through commercial or delivery-oriented engagements.	Relationship maturity, key contacts, budget signals, satisfaction, offers, meetings, and project opportunities.
Marketing	Organization involved in visibility, ecosystem development, co-branding, events, or market outreach.	Communication planning, campaign-related events, public presence, shared activities, and follow-up actions.
Supplier/ technology	Provider of technology, tools, platforms, infrastructure, or specialized services that support internal or client-facing delivery.	Technical relevance, service reliability, support agreements, supplier projects, documentation, and renewal signals.
University	Academic institution connected to research, education, student programs, events, and long-term ecosystem development.	Academic collaboration, events, grants, student-related programs, institutional contacts, and partnership renewal.

4.3 Partner lifecycle and statuses

4.3.1 Lifecycle model

The partner lifecycle describes how an external organization moves through the platform: public request, employee qualification, negotiation, activation, suspension, or closure.

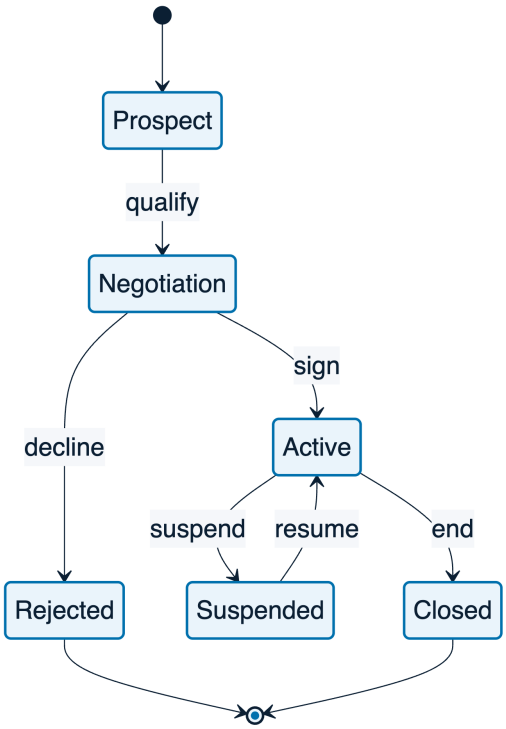


Figure 4.2: Lifecycle state diagram for partners

Table 4.3: Main lifecycle statuses for partner management

Status	Meaning	Typical transition trigger
New	Created or imported, not yet qualified.	Manual creation or accepted public request.
Under review	Under employee evaluation before a decision.	Review queue assignment or missing information.
Negotiation	Potential relationship under discussion.	Positive qualification or strategic interest.
Active	Approved and operational.	Agreement confirmed, collaboration launched, or account activated.
Suspended	Temporarily paused for risk, missing requirement, or business issue.	Low activity, unresolved issue, compliance concern, or management decision.
Closed	No longer managed as an active opportunity or collaboration.	Rejection, end of collaboration, duplicate cleanup, or explicit closure.

4.3.2 Status history for audit

Every meaningful status change is recorded in a status history entry. The audit trail stores the partner, previous state, new state, transition date, triggering action, and optional reason.

4.4 Partners CRUD and detail pages

4.4.1 Partner list and CRUD operations

The partner list is the entry point of the employee workspace. It provides search, category filters, status filters, and row actions for partner management.

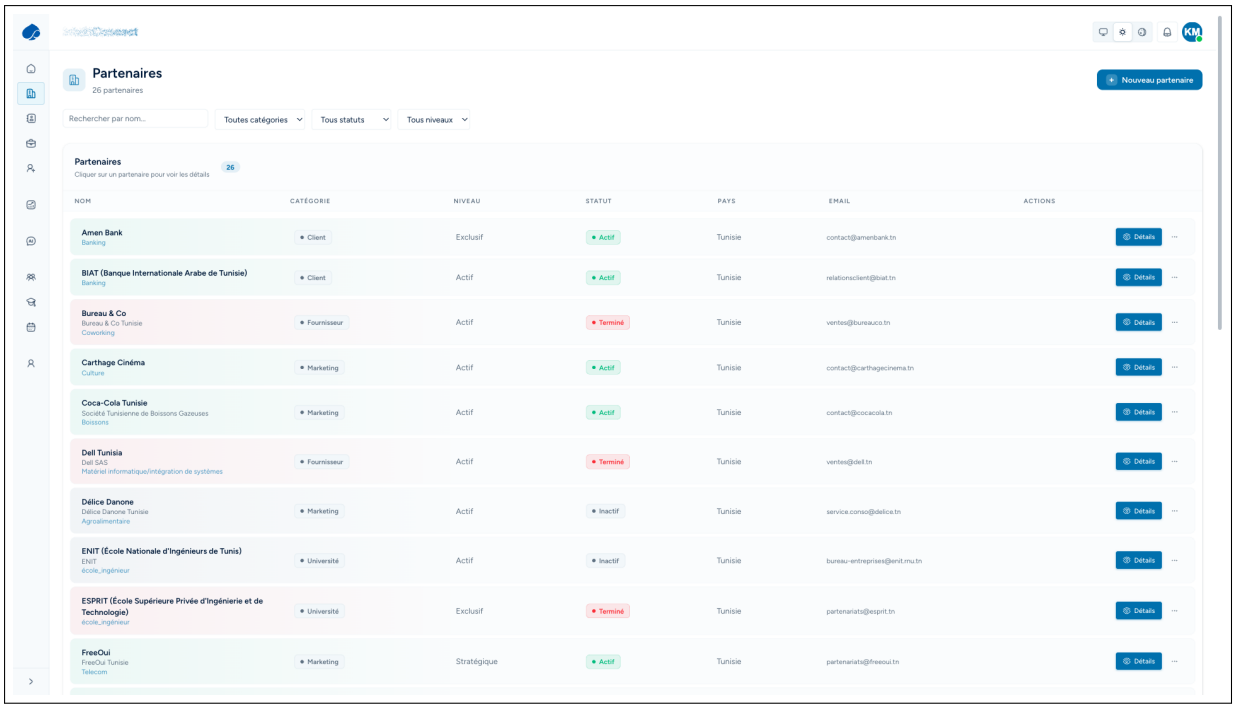


Figure 4.3: Partners list with search, filters, and row actions

The CRUD workflow follows three rules: validate essential fields before persistence, keep category and status values constrained, and show loading, success, and error feedback.

4.4.2 Partner detail page

The partner detail page consolidates one organization and keeps profile data, category, status, contacts, scheduled activity, offers or grants, documents, notifications, and status history together.

4.5 Relationship Management Entities and Data Model

Table 4.4: Core relationship management entities of the partnership module

Entity	Purpose	Main fields and relationships
Partner	Central organization record used by all modules.	Name, category, status, description, location, metadata, timestamps.
Contact	Named person linked to a partner.	Partner reference, name, role, email, phone, communication details, primary contact indicator.
Offer or grant	Collaboration proposal, commercial offer, academic grant, or support opportunity.	Partner reference, title, amount, dates, status, description, owner, documents.
Communication	Scheduled event or meeting linked to a partner.	Partner reference, type, title, date, location or channel, attendees, notes, outcome, follow-up actions.
Document	File attached to a partner or related record.	Partner reference, owner, metadata, storage key, visibility, preview information, creation date.
Notification	Actionable message for employee or partner users.	Recipient, related partner or entity, title, message, read state, priority, timestamp.
Partnership request	Public intake record submitted by an external organization.	Organization identity, requester contact, category preference, motivation, business details, review state, AI analysis.
Status history	Audit record for lifecycle transitions.	Partner reference, previous status, new status, reason, actor, transition timestamp.

4.6 Contacts directory

The contacts directory centralizes names, roles, contact details, and partner affiliation.

4.7 Offers and grants management

Offers and grants cover commercial collaboration, service opportunities, or technical support for customer and supplier/technology partners, and academic support, program fi-

nancing, sponsorship, or institutional collaboration for university partners. The interface captures title, description, partner, owner, amount when relevant, dates, and status, while offer status remains independent from partner status.

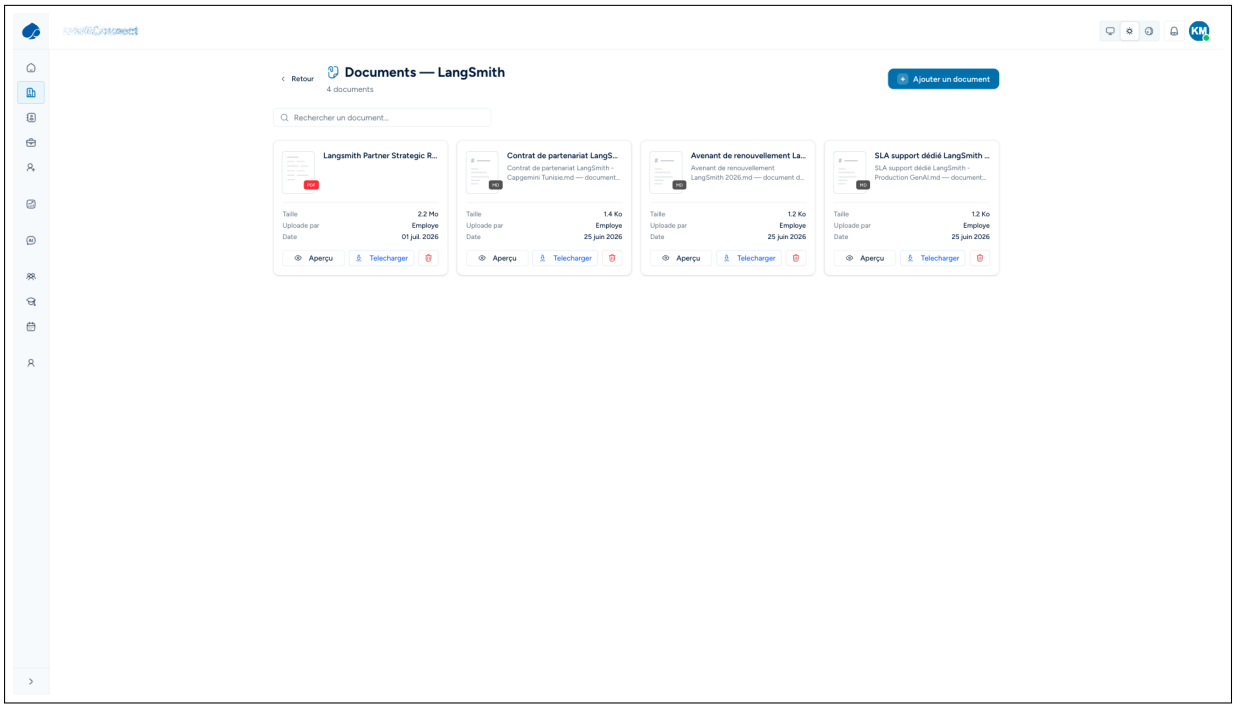


Figure 4.4: Documents and offers management views

4.8 Communications: scheduled events and meetings

Communications are scheduled events and meetings. The platform records date, type, participants, and expected follow-up. Events can represent workshops, academic sessions, ecosystem activities, or business reviews; meetings can represent operational follow-ups, negotiation sessions, technical alignment, or account reviews.

4.9 Partner documents

4.9.1 Upload, download, preview, and ownership

The platform supports upload, download, preview, and ownership for agreements, presentations, reports, specifications, event material, and administrative files while preserving the owning actor, the partner relationship, file metadata, and the creation date.

4.9.2 Document governance

Governance keeps file metadata separate from the partner record, makes visibility and ownership explicit to reduce accidental exposure, and shares one document identity across preview and download actions.

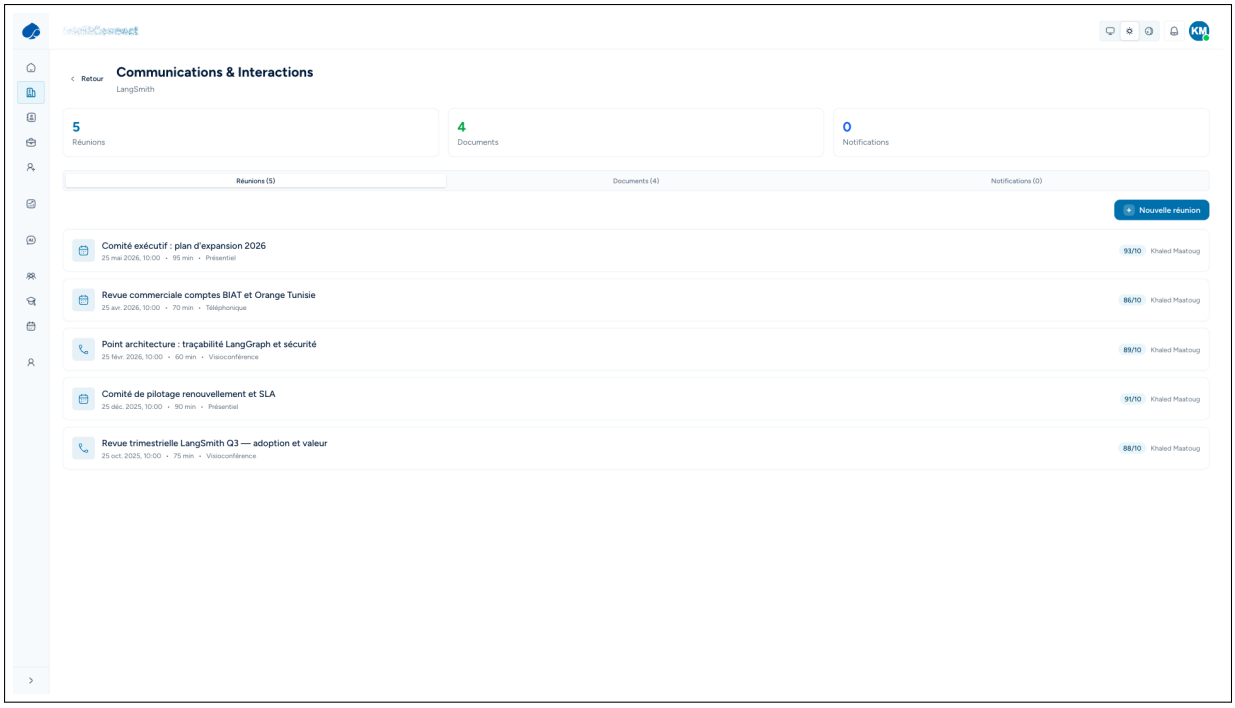


Figure 4.5: Partner communications: scheduled events and meetings

4.10 Notifications

Notifications help users react to changes without monitoring every page. They can be generated for a public request, document update, upcoming meeting, status change, offer decision, or assigned action, and they store the recipient, message, related entity, read state, and timestamp.

4.11 Public partnership and adherence request system

4.11.1 Public request intake

The public website includes a request path where an external organization can submit a partnership or adherence request. The intake captures identity, contact details, requested category, motivations, and business context. The form stays separate from the authenticated employee portal, so employees decide whether each submission becomes an internal partner record.

4.11.2 Review queue

Incoming requests are collected in a review queue. Employees inspect each request, review the AI-assisted compatibility analysis, then accept or reject it. Acceptance can create or update a partner record and initialize its lifecycle status; rejection preserves a decision trace. Figure 4.6 models this end-to-end business process across the applicant, the system, the reviewer, and the outcome, while Figure 4.7 details the same flow as a technical

sequence.

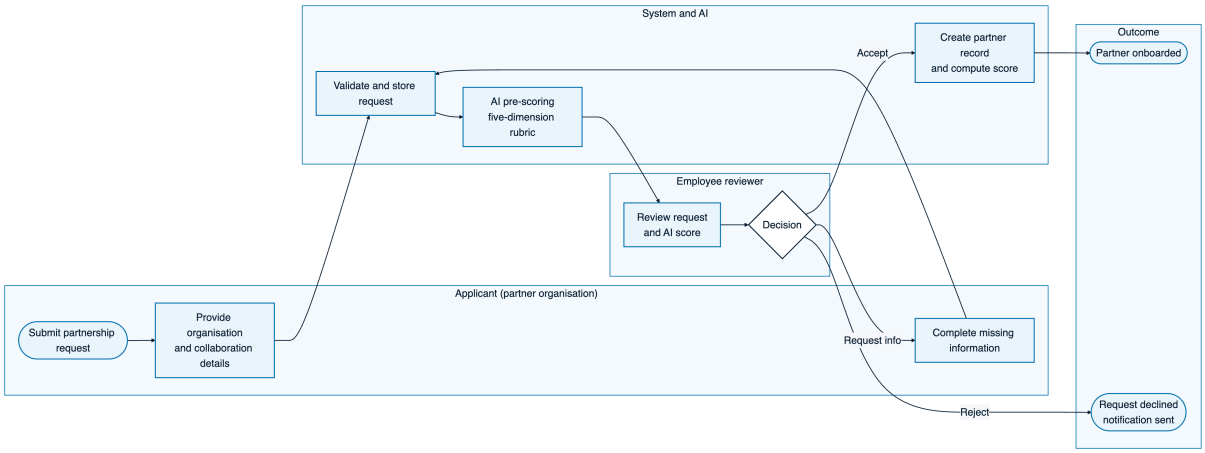


Figure 4.6: Business process (BPMN) of the partnership request-to-onboarding workflow

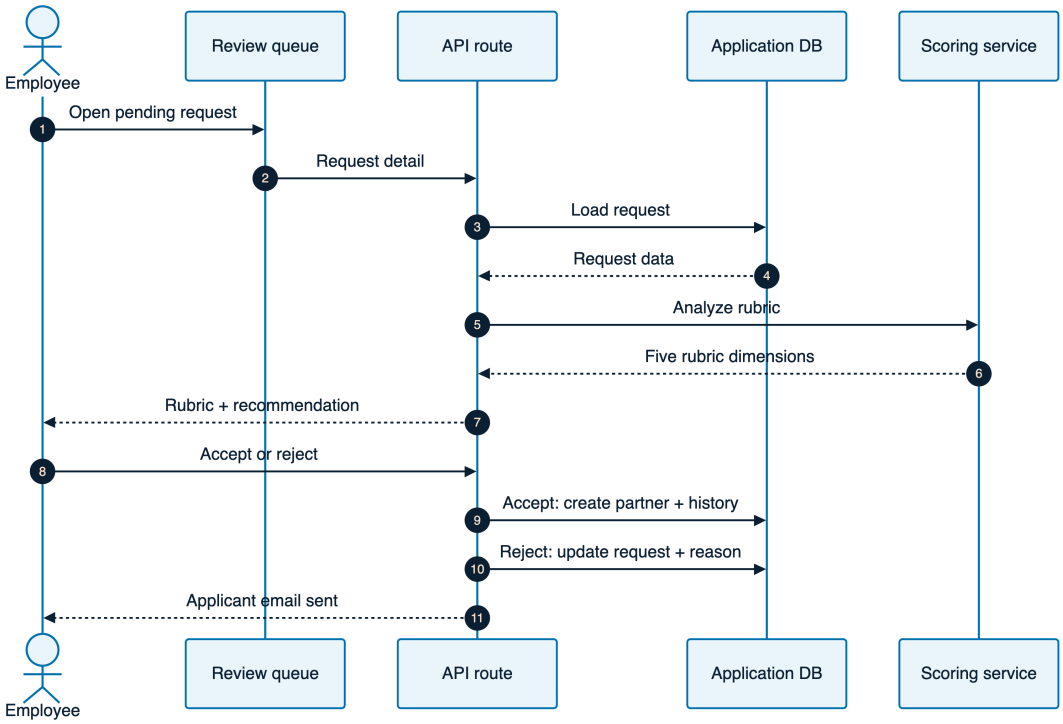


Figure 4.7: Sequence for public partnership request analysis and review

4.12 AI-assisted compatibility analysis for requests

Table 4.5: Compatibility scoring dimensions for public partnership requests

Dimension	Output	Interpretation
`dataCompleteness`	Score and explanation	Measures whether the request contains enough identity, contact, category, motivation, and business information to support a serious review.
`strategicFit`	Score and explanation	Evaluates alignment with Capgemini Engineering Tunisie priorities, expected partnership value, and relevance to the requested category.
`reliability`	Score and explanation	Estimates whether the organization appears credible, reachable, and operationally stable based on the submitted information.
`scalePotential`	Score and explanation	Assesses whether the relationship can grow beyond a single action into repeat collaboration, broader ecosystem value, or long-term contribution.
`categoryBoost`	Score and explanation	Applies category-aware adjustment when the requested category has strong contextual indicators, for example technical complementarity, academic relevance, market visibility, or commercial opportunity.

4.13 Portals and Role-Differentiated Dashboards

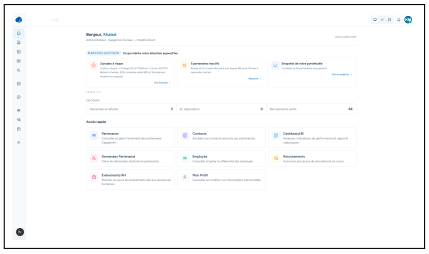
The **public website** is limited to the partnership value proposition and the application form. The **employee portal** is role-differentiated, with a daily briefing and five internal roles: administrator for accounts and audit trails; manager for lifecycle decisions and project health; commercial user for relationships, contacts, and offers; analyst for BI dashboards, scoring pages, and the AI assistant; and human-resources user for the recruitment pipeline. Figure 4.8 shows several internal operational views.

The **partner portal** is self-service. External partners manage their own data and performance; the ESPRIT university partner in Figure 4.9 creates internship and work-study offers, co-organizes events, follows its student-recruitment pipeline, manages documents and contacts, and reads a home dashboard that benchmarks its indicators against university-category averages.

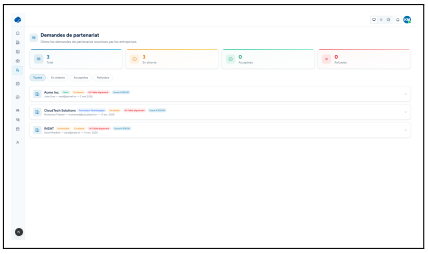
4.14 Functional validation scenarios

The module can be validated through these scenarios:

- an employee creates a partner, assigns a category, changes its status, and checks the status history;



Employee home: role-aware daily AI briefing and alerts

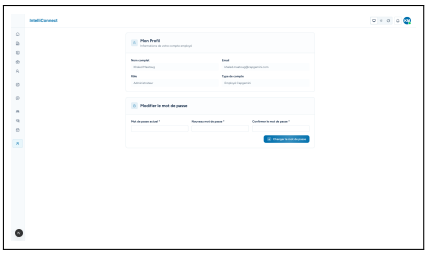


Request review queue with AI prescore

Partner contacts directory

Events with budget and attendance

Auditable partnership status history

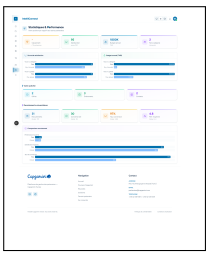


Employee profile and password

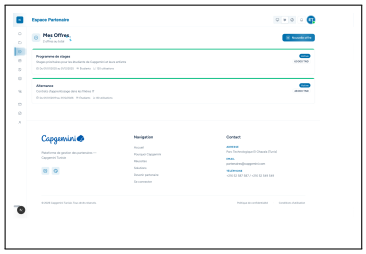
Figure 4.8: Role-differentiated employee operational views.

- an employee searches the partners list, applies category and status filters, and opens a partner detail page;
- an employee adds a contact and sees it on the partner page and in the global contacts directory;
- an employee creates an offer or grant, updates its state, and checks that the partner relationship remains unchanged unless explicitly modified;
- an employee schedules a meeting or event and sees it in the partner communications view;
- an authorized user uploads a partner document, previews it, downloads it, and verifies ownership metadata;
- a public organization submits a partnership or adherence request, which appears in the review queue;
- an employee reviews the AI-assisted compatibility dimensions and accepts or rejects the request;
- a notification is generated for a relevant action and can be marked as read.

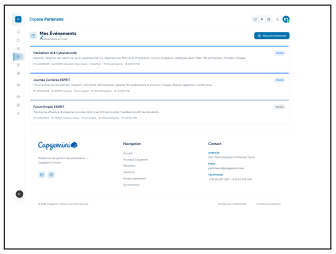
Partner home with category benchmark



Own statistics vs category averages

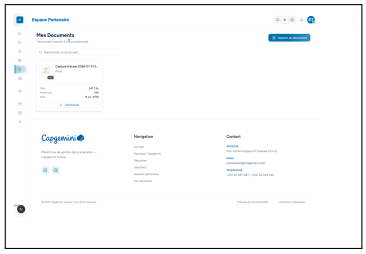


Partner creates its own offers

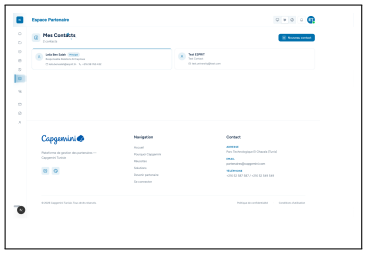


Co-organized events

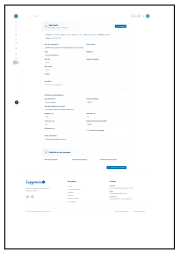
Student-recruitment pipeline



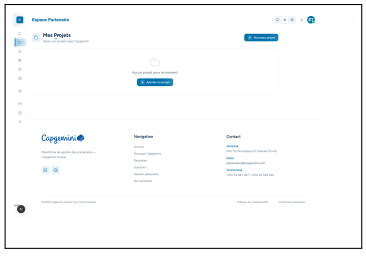
Partner documents



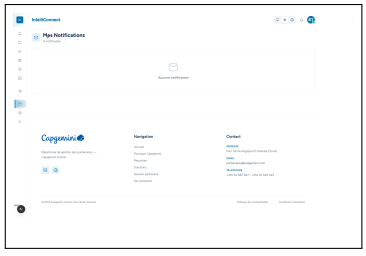
Partner contacts



Partner organization profile



Partner projects (empty state)



Notification center (empty state)

Figure 4.9: Partner self-service portal (ESPRIT university partner).

4.15 Conclusion

This chapter described the partnership lifecycle and relationship-management module of IntelliConnect. It covers CRUD, detail pages, contacts, offers and grants, communications, documents, notifications, public request intake, review queue management, AI-assisted compatibility analysis, and status history for customer, marketing, supplier/technology, and university partners. The result is a structured, searchable, categorized, and auditable foundation for later analytics and AI services.

Chapter 5

Project Delivery, Operations and BI

Introduction

This chapter covers the operational delivery layer of IntelliConnect. It connects follow-up, milestones, work allocation, documents, dashboards, and report generation, then closes with the Human Resources and employee area as an integration point for employee statistics, internal events, and staffing context.

5.1 Sprint and Module Objective

The objective of this module was to deliver an operational cockpit for partnership execution, with a coherent flow from project creation to monitoring, milestone planning, task tracking, and management-ready analytics.

- **Project portfolio control**: list partner projects with status, health, progress, and budget indicators.
- **Project execution detail**: expose milestones, mini tasks, team allocations, and project documents.
- **Decision-support analytics**: consolidate operational data into a BI dashboard backed by a separate data warehouse.
- **People and HR readiness**: expose employee statistics and events, then define the integration path with the future Capgemini HR/recruitment sister project.

5.2 Module Backlog

Table 5.1: Backlog for the project delivery, BI and HR-readiness module

ID	User story	Acceptance criteria	Priority
B5.1	As an employee, I can view a portfolio of active and archived projects.	The list shows partner, project type, health, progress, budget usage, schedule status and quick access to the detail page.	High
B5.2	As a manager, I can open a project detail page.	The page presents summary metrics, timeline, health explanation, linked partner context and execution sections.	High
B5.3	As an operational user, I can track milestones.	Milestones have labels, dates, status and a visible relation to project progress.	High
B5.4	As an operational user, I can manage small execution tasks.	A lightweight task list supports ownership, completion status and blocked items without becoming a full external project-management tool.	Medium
B5.5	As a manager, I can see team allocations.	Assigned employees, roles, allocation rates and availability context are visible in the project page.	High
B5.6	As an employee, I can attach or consult project documents.	Documents are linked to the project and remain accessible from the execution workspace.	Medium
B5.7	As an analyst, I can identify weak project health signals.	The system combines delay, budget, progress, milestone and staffing signals into readable health analysis.	High
B5.8	As an analyst, I can consult BI charts from warehouse data.	BI indicators are generated from a dedicated warehouse database and aggregated on the server before rendering.	High
B5.9	As a manager, I can export reports to PDF.	Generated report content can be opened in the report workspace and exported as a PDF deliverable.	Medium
B5.10	As an HR user, I can consult employee and event signals.	The HR area exposes employee counts, roles, events and readiness for future integration.	Medium

ID	User story	Acceptance criteria	Priority
B5.11	As a university-partnership owner, I can treat student recruitment and CDI conversion data as partner signals.	Academic partnership performance can include internships, student recruitment operations and CDI conversions without mixing them with generic HR administration.	Medium

5.3 Project Portfolio

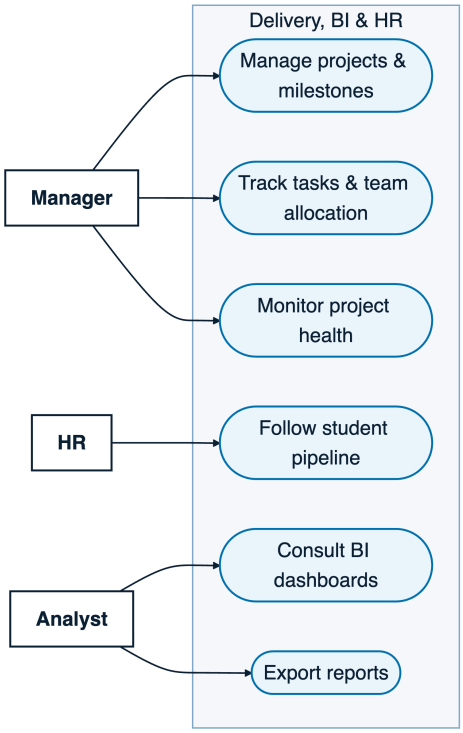


Figure 5.1: Use cases of the delivery, business-intelligence, and human-resources module.

- **Health and progress**: on track, under observation, or at risk, with completion percentage derived from project progress and milestones.
- **Budget, schedule, and access**: consumption versus plan, start date, target date, delay, overdue indicators, partner organization, category, and quick navigation to the detail page.

5.4 Project Detail Page

5.4.1 Health, Progress, Budget and Schedule

- **Health, progress, budget, and schedule**: qualitative risk view, completion, timeline drift, financial drift, start dates, target dates, and delays.

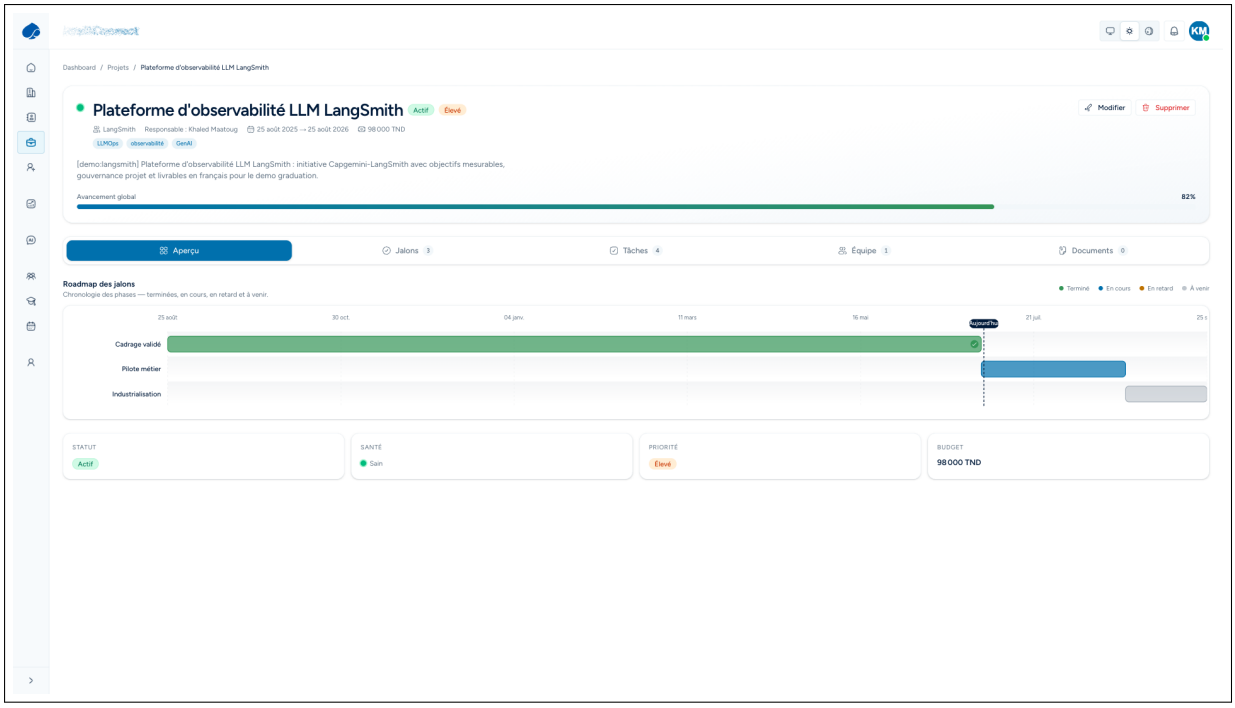


Figure 5.2: Project detail workspace.

5.4.2 Milestones

- milestone title, description, planned date, and deadline;
- status such as planned, in progress, completed, or blocked, plus contribution to the overall progress view and health signals when key work is late.

5.4.3 Mini Task System

- a short title, owner, and completion state;
- a due date when relevant, plus a blocked flag or local note for follow-up.

5.4.4 Team Allocations

- staffing sufficiency, delay cause, and overload risk.

Figure 5.3: Project health computed from delivery signals.

5.4.5 Project Documents

5.5 Project Health Analysis Signals

Table 5.2: Project health analysis signals

Signal	Detection logic	Operational interpretation
Schedule delay	Target date is exceeded or projected completion is later than the planned end date.	The project may require escalation, partner alignment or replanning.
Progress gap	Completion percentage is lower than expected for the elapsed timeline.	Execution is slower than planned even if the final deadline has not yet passed.
Budget overrun	Consumed or forecast budget exceeds the planned budget threshold.	Financial governance is needed before continuing at the same pace.
Blocked milestone	A milestone with high importance is marked blocked or remains incomplete after its due date.	A critical dependency is preventing normal delivery.
Low allocation	Assigned team capacity is insufficient compared with project priority or workload.	Staffing adjustment may be required.
Missing documentation	Key delivery or validation documents are absent.	Traceability and acceptance may become weak during review.
Partner dependency	A task or milestone waits for partner input for too long.	The issue is external and should be handled through partner communication.
Repeated status drift	Health status has degraded multiple times over the project lifecycle.	The project has systemic instability rather than a one-time incident.

5.6 BI Dashboard and Data Warehouse Architecture

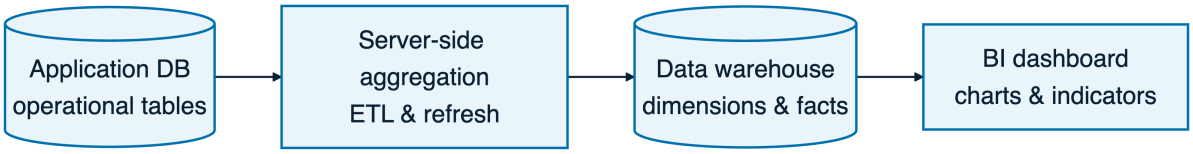


Figure 5.4: Application database to data warehouse flow.

5.6.1 Server-Side Aggregation

- partner distribution by category;
- project health distribution;
- budget by partner category and progress by project status;
- event and activity trends, plus university student recruitment, CDI conversion, and employee allocation indicators;

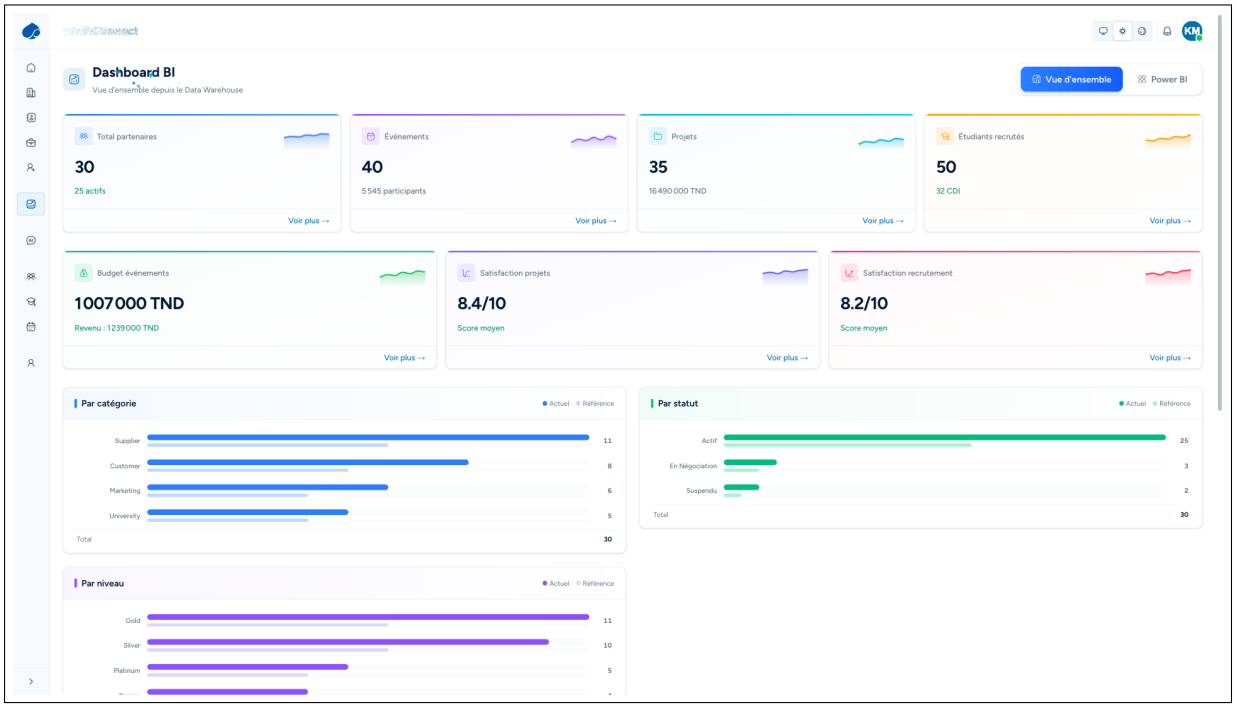


Figure 5.5: Business-intelligence dashboard with warehouse-backed charts.

5.6.2 Warehouse Dimensions and Facts

Table 5.3: Data warehouse dimensions and facts

Warehouse object	Type	Analytical role
`dim_partner`	Dimension	Stores partner identity, category, level, status and segmentation fields for portfolio analysis.
`dim_project`	Dimension	Stores project identity, type, status, planned dates and partner relation.
`dim_employee`	Dimension	Stores employee role, department and availability attributes used for allocation analysis.
`dim_date`	Dimension	Standardizes day, month, quarter and year grouping for trends.
`fact_project_health`	Fact	Stores health, progress, schedule delay, budget usage and milestone completion snapshots.
`fact_partner_activity`	Fact	Aggregates meetings, events, offers, documents and operational interactions by partner and date.
`fact_budget`	Fact	Tracks planned and consumed budget by project, partner category and period.
`fact_university_talent`	Fact	Stores university student recruitment volume, internships, accepted offers and CDI conversions as partnership-performance signals.
`fact_employee_allocation`	Fact	Aggregates employee allocation rate, project load and availability over time.

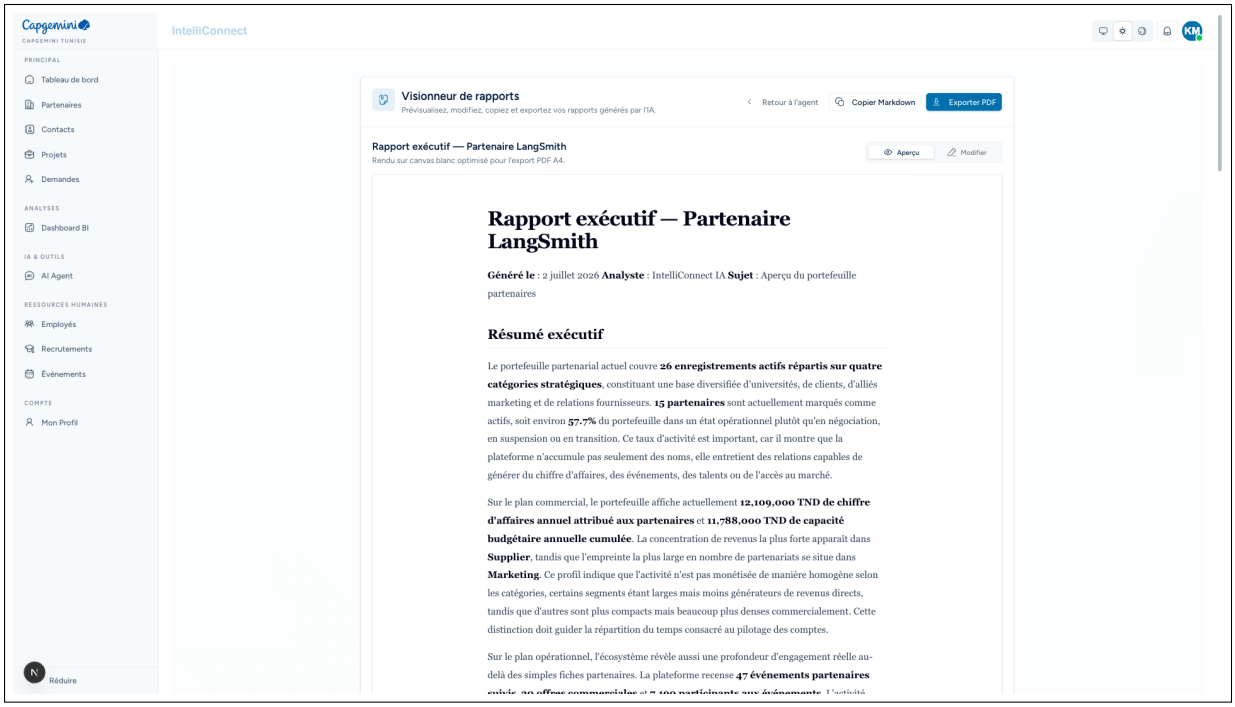
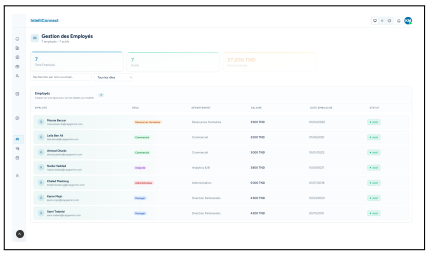


Figure 5.6: Reports workspace with markdown preview and PDF export.

5.7 HR and Employee Module as an Integration Point

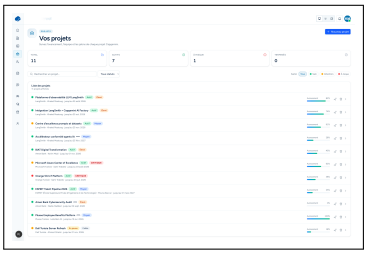
5.7.1 Current Employee Statistics and Events



Employee roster with role and payroll KPIs

Student-recruitment pipeline and CDI conversions

HR events organizer



Project portfolio with health and progress

Figure 5.7: Human-resources views and the project portfolio list.

5.7.2 University Student Recruitment and CDI Conversion Signals

5.7.3 Future Integration with Capgemini's HR/Recruitment Sister Project

Table 5.4: HR and recruitment integration roadmap

Phase	Scope	Expected result
Current foundation	Employee roster, employee statistics, internal events and project allocations.	IntelliConnect can support delivery staffing and basic people-related dashboards without becoming a full HR system.
Controlled data exchange	Import aggregated university student recruitment indicators and CDI conversion counts from the HR/recruitment sister project.	Academic partnership performance becomes measurable through trusted talent-outcome signals.
Staffing intelligence	Use employee skills, availability and allocation load to recommend staffing options for partner projects.	Managers can identify suitable internal contributors with lower manual effort.
Strategic BI alignment	Combine partner, project, budget, event, university student recruitment and CDI conversion facts in the data warehouse.	BI dashboards can correlate partnership activity with delivery and talent outcomes.
Governance and audit	Define access rules, synchronization logs and data minimization policies for exchanged HR/recruitment data.	The integration remains secure, explainable and limited to business-relevant indicators.

5.8 Conclusion

This chapter presented the delivery and operations layer of IntelliConnect. The project portfolio gives managers an overview of active initiatives, while the project detail page provides the evidence needed for execution: health, progress, budget, schedule, milestones, mini tasks, team allocations, and documents. The BI dashboard uses a separate data warehouse and server-side aggregation, reports complete the decision-support flow, and

the HR module remains a light integration point with Capgemini's HR/recruitment sister project for university student recruitment and CDI conversion data.

Chapter 6

Agentic AI Harness and Generative Interface

Introduction

This chapter presents the AI module of IntelliConnect as a decision-support system for Capgemini Tunisia. It helps employees explore the partnership portfolio, compare partners, assess delivery risk, inspect evidence, and generate executive reports. The core choice is an **agentic AI harness**. The assistant clarifies uncertainty, declares a method, plans work, calls typed tools, inspects results, visualizes evidence, and returns a sourced answer.

6.1 Sprint Objective and Module Backlog

6.1.1 Objective of the AI module

This sprint turned IntelliConnect into an intelligent partnership cockpit. The AI layer combines partners, projects, documents, metrics, and reports in one guided flow. The module answers partnership questions with real platform data, guides complex analysis through methodology and planning, calls typed tools for partner and project intelligence, visualization, reporting, and RAG, streams reasoning with plan progress and source links, and produces exportable executive reports. It is accessed from the employee portal at `/dashboard/agent` and complements the structured screens with a higher-level interface for multi-step questions.

6.1.2 Sprint backlog

Table 6.1: Backlog of the agentic AI module

ID	Theme	User Story	Status
1	Agent workspace	As an employee, I can open a dedicated AI workspace from the dashboard and start an analytical conversation.	Completed
2	Streaming runtime	As a user, I receive streamed responses instead of waiting for a full blocking answer.	Completed
3	Reasoning visibility	As a user, I can inspect the assistant's plan, reasoning trace, and methodological choices.	Completed
4	Tool orchestration	As a user, my complex request can trigger multiple validated tools before the final synthesis.	Completed
5	Partner analytics	As a user, I can ask questions about partners, scores, categories, activity, churn risk, and recommendations.	Completed
6	Project analytics	As a user, I can inspect delivery health, risk signals, delays, staffing fit, and critical paths.	Completed
7	Visual outputs	As a user, I receive dynamic bar charts, line charts, pie charts, and tables inside the answer.	Completed
8	Report generation	As a user, I can generate an executive report and export it as a PDF.	Completed
9	RAG evidence	As a user, I can search partner and project documents and receive cited excerpts with deep links.	Completed
10	Guardrails	As a product owner, I need timeouts, no fabricated data, French user-facing output, and source-backed conclusions.	Completed

6.2 Agentic AI vs. a Simple Chatbot

A simple chatbot returns fluent text, but it can stay disconnected from the operational system and lack current data, methodology, intermediate work, or traceable sources. That is insufficient for a partnership-management platform where decisions depend on budgets, project status, activity history, scores, documents, and user roles. The IntelliConnect assistant instead searches before concluding, asks for clarification when the request is under-specified, declares its methodology, creates a plan, calls tools, synthesizes evidence, visualizes results, and attaches sources.

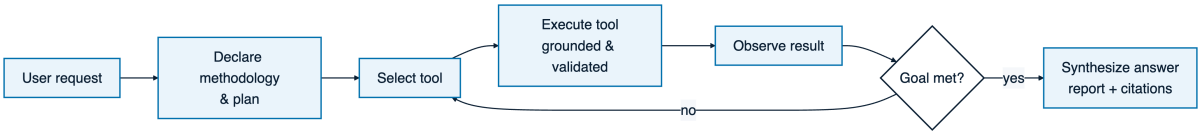


Figure 6.1: The agentic reasoning loop of the IntelliConnect assistant.

Table 6.2: Difference between a simple chatbot and the IntelliConnect agentic harness

Dimension	Simple chatbot	IntelliConnect agentic harness
Data grounding	Generates text mainly from model context.	Calls typed tools connected to platform data and documents.
Execution structure	Usually single-turn text completion.	Multi-step workflow with plan, tool calls, synthesis, and sources.
Methodology	Often implicit or absent.	Declares an analytical method before deep analysis.
Uncertainty handling	May answer even when the prompt lacks detail.	Uses clarification when scope, entity, period, or output format is ambiguous.
Visual output	Mostly text.	Interleaves charts, tables, report cards, source links, and reasoning traces.
Evidence	May not expose where facts came from.	Uses tool outputs, RAG excerpts, and deep links to product pages.
Product fit	Generic assistant behavior.	Specialized partnership-intelligence behavior aligned with IntelliConnect.

6.3 Harness Engineering from the System Prompt

The quality of an agent depends less on the raw language model than on the harness around it, including the operating contract, tool set, grounding rules, and execution loop. This view, that the harness matters more than the model itself [40], guided the design of the IntelliConnect assistant.

6.3.1 System prompt as an execution contract

The system prompt defines the assistant's operating contract before any tool call occurs. The key rule is **SEARCH BEFORE CONCLUDING**, which prevents synthesis before the assistant has inspected the available platform data. The harness requires seven recurring behaviors: **Search before concluding**, **Clarify** when scope is unclear, **Declare methodology**, **Create a plan**, **Run tools** for retrieval, computation, visualization, report creation, and RAG, **Synthesize** without inventing facts, and **Visualize and cite**

sources near the supporting section.

6.3.2 Agent orchestration architecture

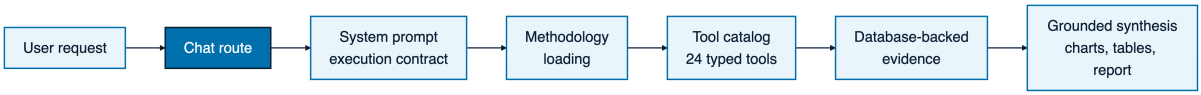


Figure 6.2: Agent orchestration from user request to grounded synthesis

The orchestration runs from the chat interface to the server route that hosts the AI runtime. The runtime injects the system prompt, loads the tool catalog, and streams the answer back to the UI while allowing tool calls during generation. Each tool validates its input, queries the relevant service or database-backed function, and returns structured results for text, charts, tables, source links, or report cards. The orchestration separates three responsibilities: **The model** chooses the method and tool sequence, **the tools** perform validated access to real application data, and **the UI renderer** turns tool results into an interactive interface.

6.4 AI SDK Runtime

6.4.1 Streaming execution

The runtime is implemented with the Vercel AI SDK. `streamText` streams tokens, tool-call events, reasoning information, and final content, while `toUIMessageStreamResponse` converts the stream into a UI-compatible event feed. The runtime configuration includes `sendReasoning: true`, which supports the plan HUD and reasoning trace by exposing the assistant's intermediate analytical path. It also defines `MAX_AGENT_STEPS = 1000`; executive analysis can require several tool calls, so the value is balanced by tool-level timeouts and strict data-grounding rules.

6.4.2 NVIDIA-compatible chat provider

The model provider is configured through an NVIDIA-compatible chat interface. The application can connect to a chat-completion endpoint that follows an OpenAI-style protocol while remaining compatible with NVIDIA-hosted or NVIDIA-routable models. In practice, `streamText` starts model execution, `toUIMessageStreamResponse` sends the structured UI stream, `sendReasoning: true` exposes reasoning events, `MAX_AGENT_STEPS = 1000` keeps long multi-tool analysis possible, and the NVIDIA-compatible chat provider supplies the model.

6.5 Method and Skill Loading

6.5.1 Purpose of analytical method loading

The assistant does not treat every question as the same generic task. It auto-loads analytical methodologies that guide how the answer should be structured and provide reusable patterns for decomposition, comparison, scoring, benchmarking, and funnel analysis. Exactly nine analytical methodologies are auto-loaded for this module and are listed in Table 6.3.

Table 6.3: Auto-loaded analytical methodologies

No.	Methodology	Analytical Purpose	Typical Output
1	Margin Waterfall	Decomposes margin movement into revenue, delivery cost, staffing, delay, discount, and operational effects.	Waterfall explanation, variance table, executive summary.
2	Best-in-Class + Root Cause	Compares top performers with lower performers, then traces the operational drivers behind the gap.	Benchmark table, root-cause narrative, priority actions.
3	COGS Comparison	Compares cost of goods or delivery effort across partners, projects, categories, or periods.	Cost comparison table, bar chart, efficiency interpretation.
4	Revenue Decomposition	Breaks revenue movement into volume, price, project mix, partner category, and period effects.	Revenue bridge, line chart, segment commentary.
5	Peer Benchmark	Compares one partner or project against a peer group with normalized indicators.	Benchmark matrix, ranking, gap analysis.
6	Activity-Decay Churn	Detects churn risk by measuring declining interactions, events, meetings, offers, KPIs, and document activity.	Risk score, warning signals, retention plan.
7	Strategic-Fit Scoring	Scores alignment with partnership category, status, budget, satisfaction, activity, track record, and strategic fit.	Score table, radar or bar visualization, recommendation.
8	Vendor Performance Index	Aggregates supplier delivery signals such as project health, delay exposure, budget pressure, and support quality.	Supplier index, risk segmentation, corrective actions.
9	Recruitment Funnel	Measures academic or HR-related flow from sourcing volume to conversion and retention outcomes.	Funnel table, conversion ratios, improvement actions.

6.6 Tool Calling Layer

6.6.1 Typed tools with Zod schemas

The harness exposes tools through the AI SDK `tool()` pattern. Each tool defines a description, a Zod input schema, and a server-side execution function. The Zod schema constrains tool arguments before business logic executes and prevents malformed natural-language intent from becoming unsafe server input. The general tool shape is a precise tool name, a description, a Zod input schema, a bounded execution function, and a safe result format.

6.6.2 Complete tool catalog

The AI harness exposes exactly twenty-four tools, grouped into seven categories. Table 6.4 lists the full catalog.

Table 6.4: Tool catalog exposed to the agentic harness

No.	Group	Tool	Purpose
1	Meta	`declareMethodology`	Declares the analytical method selected for the request before execution.
2	Meta	`createPlan`	Creates a visible multi-step plan for complex analysis.
3	Meta	`askClarification`	Requests missing scope, entity, metric, period, or output details.
4	Partner intelligence	`queryPartners`	Searches partners by name, category, status, activity, score, or portfolio filters.
5	Partner intelligence	`getPartnerDetails`	Retrieves a detailed partner profile for analysis and deep links.
6	Partner intelligence	`queryAnalytics`	Retrieves aggregate metrics for partner activity, budgets, satisfaction, and trends.
7	Partner intelligence	`scorePartner`	Computes or retrieves strategic scoring for one partner or a batch of partners.
8	Partner intelligence	`predictChurn`	Estimates partnership churn risk from activity, satisfaction, and relationship signals.
9	Partner intelligence	`recommendPartners`	Produces partner recommendations based on category, objective, score, and fit.
10	Portfolio	`summarizePartnerPortfolio`	Summarizes the whole portfolio by category, status, score, budget, and activity.
11	Portfolio	`getCategoryBenchmarks`	Provides category-level benchmarks for peer comparison and strategic positioning.
12	Portfolio	`getPartnerActivityTimeline`	Builds a timeline of events, meetings, offers, documents, and KPI interactions.
13	Project intelligence	`analyzeProjectHealth`	Evaluates project health using progress, budget, milestones, tasks, and risk indicators.
14	Project intelligence	`identifyAtRiskProjects`	Lists projects exposed to delay, budget pressure, weak progress, or operational blockers.
15	Project intelligence	`forecastProjectDelay`	Forecasts potential delay using progress, milestone, and task completion signals.
16	Project intelligence	`recommendStaffing`	Recommends internal staffing options based on skills, availability, and fit.
17	Project intelligence	`findCriticalPath`	Identifies the task or milestone chain most likely to affect delivery timing.
18	Project intelligence	`crossEntityAnalysis`	Combines partner and project context to produce joint operational insights.
19	Visualization	`createBarChart`	Creates a bar chart for ranked scores, category comparison, or risk levels.
20	Visualization	`createLineChart`	Creates a line chart for KPI trends, activity evolution, or period comparison.
21	Visualization	`createPieChart`	Creates a pie chart for category distribution, activity mix, or status share.
22	Visualization	`createTable`	Creates a structured table for comparisons, recommendations, source summaries, or action plans.
23	Reporting	`generateReport`	Generates an executive report artifact that can be previewed and exported as a PDF.
24	RAG	`searchDocuments`	Performs semantic document search and returns cited excerpts with source metadata.

6.6.3 Tool-call sequence

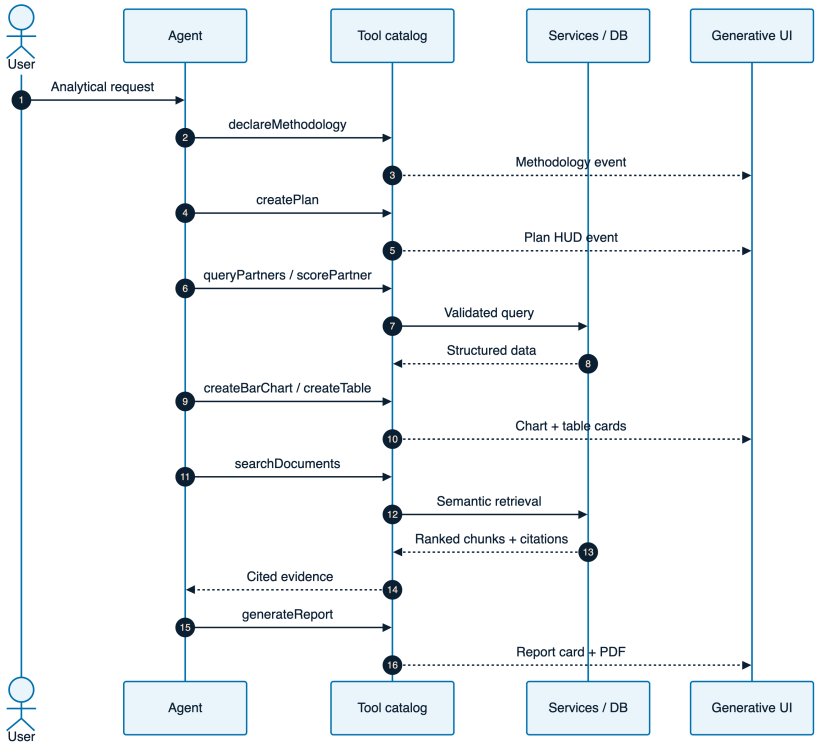


Figure 6.3: Sequence of method declaration, plan creation, tool calls, visualization, and synthesis

A typical executive report request follows this sequence: first, the model calls `declareMethodology` to state the analytical frame. Second, it calls `createPlan` so the user sees the expected steps. Third, it retrieves the relevant partner or portfolio data through partner-intelligence tools. Fourth, it invokes visualization tools to create charts and tables near the related analysis section. Fifth, it uses `searchDocuments` if the prompt asks for document-backed clauses or evidence. Finally, it calls `generateReport` when an exportable executive artifact is requested.

6.7 Generative User Interface

6.7.1 From streamed text to rich analytical workspace

The UI is generative because the response is neither a static page nor a plain text transcript. The assistant can stream text, update a plan HUD, display reasoning trace events, render charts and tables, attach source links, and produce a report preview card. Supported elements include bar, line, and pie charts; tables; report preview and PDF export; a plan HUD; reasoning trace; and source links.

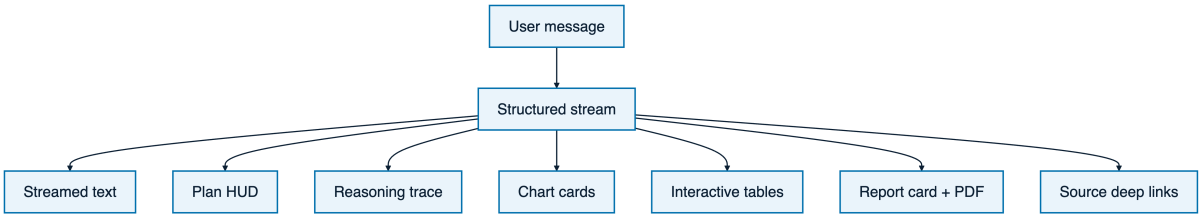


Figure 6.4: Generative UI flow from streaming events to rendered analytical components

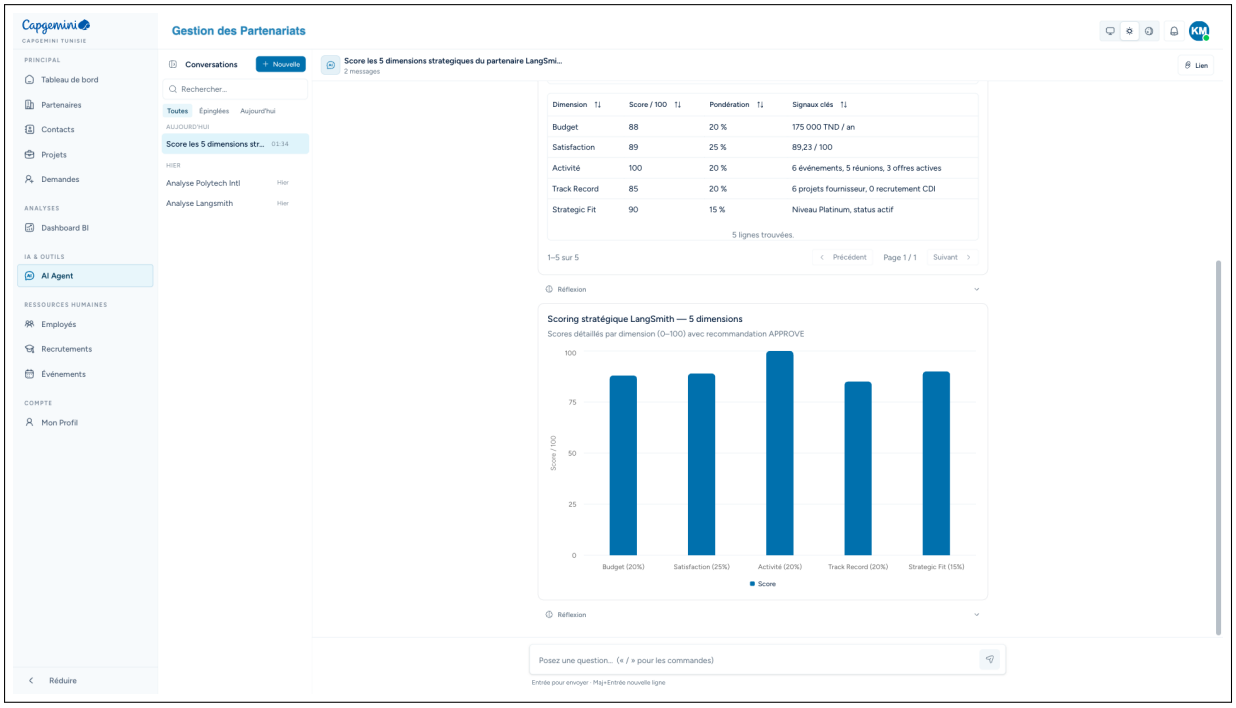


Figure 6.5: Agent chat workspace

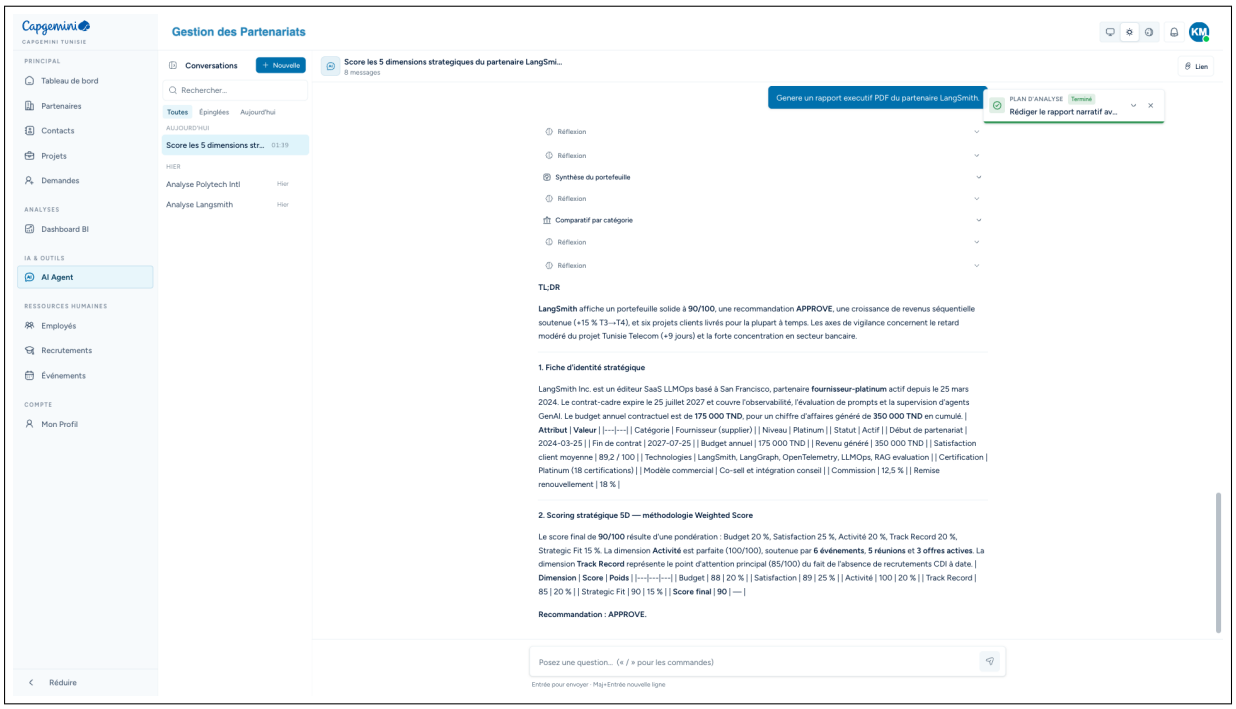


Figure 6.6: Plan HUD and reasoning trace

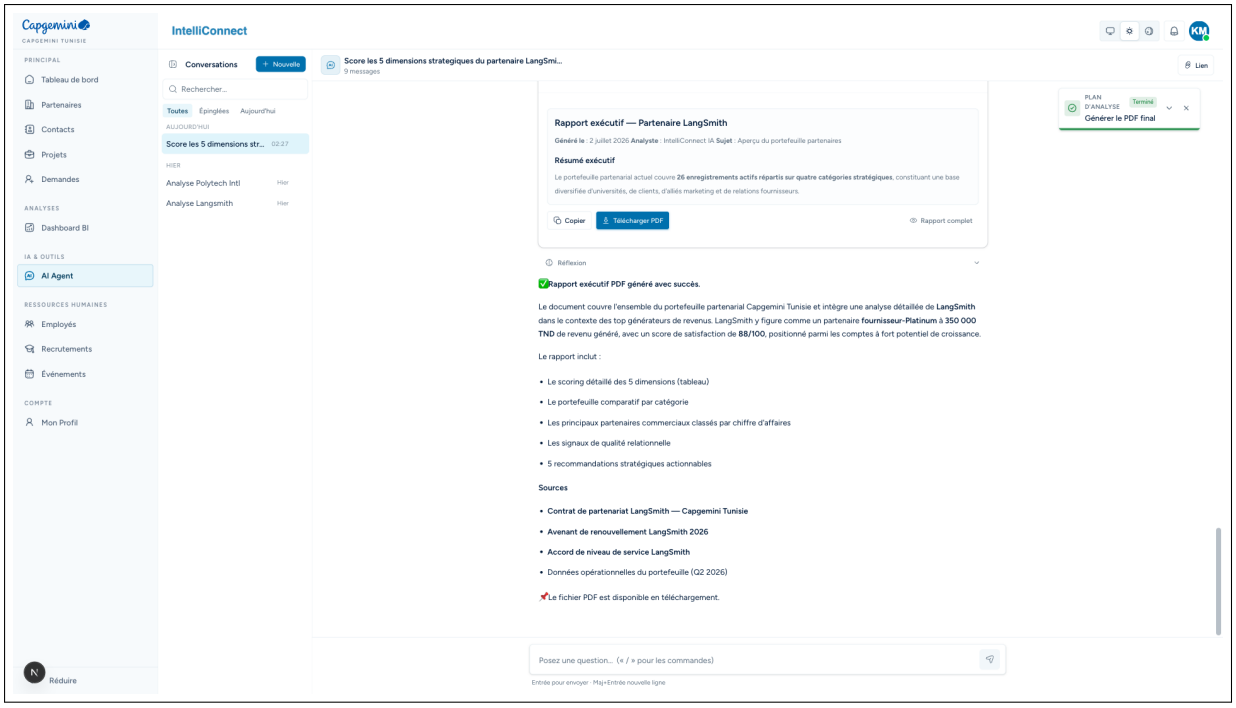


Figure 6.7: Generated report card

6.7.2 Generative UI flow

6.7.3 Implementation

6.8 Guardrails and Product Constraints

6.8.1 Runtime and data guardrails

The AI module includes guardrails at multiple levels. Tool calls use timeouts so a slow query or remote model step does not block the entire chat session. The agent is instructed not to fabricate data. If a tool returns no result, the assistant must say that no data was found and propose a next step instead of filling the gap with plausible text. The platform also enforces language and navigation constraints: user-facing output is written in French, because the application UI is French, while technical implementation details remain in English in the codebase. Source links are deep links into IntelliConnect pages, such as partner profiles, scoring pages, projects, documents, reports, or BI views. The main guardrails are tool timeouts, no fabricated data, French user output, deep links, interleaved visualizations, validated inputs, structured errors, and read-oriented behavior.

6.8.2 Evidence discipline

The agent's evidence discipline is what makes it suitable for an AI Engineer chapter. The system creates a repeatable analytical loop: understand the request, choose a method, plan the work, search the available data, call bounded tools, render structured outputs, and cite sources. When the assistant compares three partners, it should retrieve the partners, compute or read their scores, build a comparative table, create a bar chart,

explain the gaps, and link to the scoring pages. When the assistant generates a document-backed report, it should use RAG to retrieve excerpts and place source references near the claims they support.

6.9 Representative End-to-End Scenario

An end-to-end request shows how the pieces combine. Suppose an employee asks for an executive analysis of a strategic supplier. The assistant first declares a method such as Strategic-Fit Scoring or Vendor Performance Index. It then creates a plan covering profile retrieval, score analysis, activity distribution, KPI trends, churn signals, document evidence, and report generation. The runtime streams this plan to the UI. Tool calls retrieve the supplier profile, scoring dimensions, portfolio benchmarks, project health, activity timeline, and relevant document excerpts. Visualization tools create a bar chart for scoring, a pie chart for activity mix, a line chart for KPI evolution, and a table for the action plan. The assistant synthesizes the results in French and adds source links to the partner profile, scoring page, project pages, and report viewer. If requested, `generateReport` produces a report card with a PDF export action.

6.10 Conclusion

The Agentic AI Harness and Generative Interface module gives IntelliConnect a high-value analytical layer for partnership management. It lets employees ask complex questions, receive grounded answers, inspect charts and tables, follow reasoning, and export reports. Architecturally, it shows how a language model can sit inside a controlled harness with typed tools, Zod validation, method loading, streaming UI events, source links, and safety constraints. The assistant searches before concluding, clarifies uncertainty, declares methodology, creates a plan, runs validated tools, synthesizes evidence, visualizes results, and cites sources. This turns natural language into a structured analytical interface for Capgemini Tunisia's partnership operations.

Chapter 7

Knowledge Retrieval, Scoring and Observability

7.1 Introduction

This chapter presents the intelligence layer that turns IntelliConnect into an evidence-driven decision support system. It combines a Retrieval-Augmented Generation (RAG) knowledge layer, deterministic and AI-assisted scoring, and observability for tracing and improving AI behaviour. The module keeps Capgemini Tunisia teams in control. It gives reviewers a repeatable way to retrieve source material, compare partners with transparent scoring rules, assess incoming requests, and inspect how the agent reached a result. Partnership decisions still rely on contracts, project documents, events, meetings, satisfaction indicators, activity history, financial signals, and strategic alignment notes. The implementation follows four principles: grounded answers linked to retrieved documents, explainable scoring built from dimensions, weights, thresholds, and reasons, observability through LangSmith traces and local tests, and reproducible validation with automated evaluation commands, a Vitest suite, and user-facing checks.

7.2 Sprint and Module Objective

The sprint objective was to deliver a complete intelligence and control layer for partnership operations. By the end of the sprint, an employee user had to search partner and project documents semantically, obtain cited AI answers, compute an explainable partner score, score incoming requests with AI assistance, and inspect the agent through observability traces and evaluation results.

7.2.1 Functional Scope

The module covers document ingestion into a vector knowledge base, semantic retrieval over stored chunks, exposure of the `searchDocuments` tool, citation rendering in agent answers and reports, hybrid scoring for existing partners and incoming requests, trace

collection with tool-span, latency, and error monitoring, prompt and version tracking through LangSmith, and evaluation feedback through a curated evaluation suite and Vitest tests.

7.2.2 Sprint Backlog

Table 7.1: Sprint backlog for knowledge retrieval, scoring, and observability

ID	User story	Acceptance criteria	Status
RAG-01	As an employee, I want partner and project documents to be indexed so that the AI agent can retrieve evidence from internal knowledge.	Documents are chunked, embedded, and stored with metadata in `documentEmbeddings`.	Done
RAG-02	As an employee, I want semantic document search so that I can find relevant clauses and operational facts without exact keyword matching.	Search uses cosine distance, returns ranked chunks, and enforces the configured result limit.	Done
RAG-03	As an employee, I want cited AI answers so that I can verify where a recommendation comes from.	The agent displays source titles, snippets, entity links, and document references.	Done
SCO-01	As a manager, I want deterministic partner scoring so that partner health can be compared consistently.	The model uses five dimensions, fixed weights, and clear decision thresholds.	Done
SCO-02	As a reviewer, I want AI-assisted request scoring so that incoming requests can be prioritised with a structured rubric.	The request score exposes dimensions, reasons, and a recommended action.	Done
OBS-01	As a technical maintainer, I want agent traces so that each generation can be inspected end to end.	LangSmith records traces, model calls, tool spans, prompt versions, latency, and errors.	Done
OBS-02	As a project owner, I want repeatable AI evaluations so that regressions can be detected before a demonstration.	A curated evaluation suite and the Vitest suite validate agent behaviour and tool contracts.	Done

7.3 RAG Knowledge Layer

The RAG layer gives the AI agent access to internal partnership knowledge while reducing unsupported answers. In IntelliConnect, the corpus includes framework agreements, renewal conditions, meeting notes, delivery documents, support commitments, project summaries, and other operational files uploaded through the partner and project modules.

IntelliConnect uses a **hybrid, agentic RAG** design. The agent decides when to retrieve, then combines deterministic structured signals from the operational database with semantic passages from the vector index. As shown in Figure 7.1, a single answer can cite an exact contractual clause while grounding figures in real database values. The retrieval gains over a purely lexical baseline are quantified later in this chapter, and the design follows the retrieval-augmented generation literature [27].

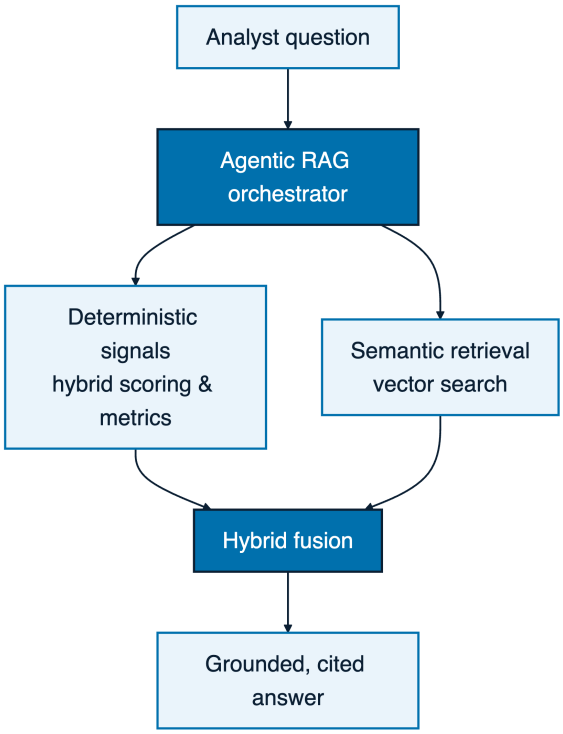


Figure 7.1: Hybrid, agentic RAG: deterministic signals fused with semantic retrieval.

7.3.1 Document Ingestion

The ingestion pipeline starts when a partner or project document reaches the platform. The file is extracted as text, normalised, linked to business metadata, and split into reusable chunks before embedding. Each chunk keeps its source document, partner or project entity, title, file name, and storage reference so the agent can show a passage with its context. The splitter is recursive, with a chunk size of **1200 characters** and an overlap of **200 characters**. It preserves larger semantic units when possible, then falls back to smaller separators. The overlap keeps local context across boundaries so split clauses stay understandable during retrieval.

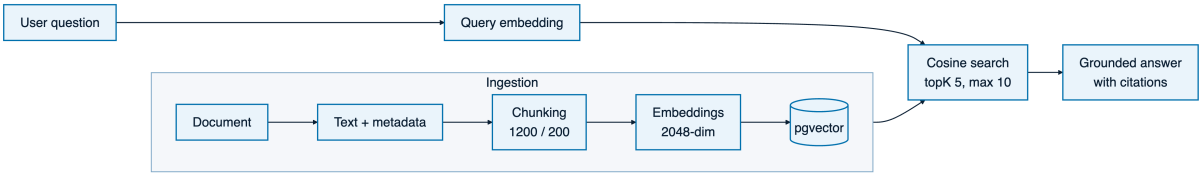


Figure 7.2: RAG ingestion and retrieval pipeline

7.3.2 Embedding and Vector Storage

Each chunk is embedded through an OpenRouter model that returns vectors with **2048 dimensions**. The vectors are stored in PostgreSQL with `pgvector`. The `documentEmbeddings` table stores the embedding, chunk text, chunk order, metadata, and entity references. Keeping `pgvector` in the same database simplifies joins with partners, projects, and documents. It also lets the implementation enforce business filters before or during retrieval, such as limiting results to one partner or project when the question carries entity context.

7.3.3 Retrieval Parameters

IntelliConnect retrieves through two complementary channels and fuses them. A dense channel embeds the query and ranks stored chunks by **cosine distance**, capturing semantic proximity even when wording differs [31]. A lexical channel scores the same chunks with **BM25** over a full-text index, which stays strong on exact terms such as clause identifiers, product names, and contractual references [32]. The two ranked lists are combined with **Reciprocal Rank Fusion** (RRF), a rank-level fusion that is robust and needs no score calibration [33], and a final **re-ranking** pass reorders the fused top candidates before they reach the agent. This hybrid, re-ranked pipeline is the semantic matching engine of IntelliConnect: the dense channel runs over `pgvector` storage, the lexical channel over a full-text index, and the fusion and re-ranking stages sit in front of the `searchDocuments` tool. The agent-facing tool exposes a default of **topK = 5** and accepts values up to **10**. This gives enough evidence for concise answers while avoiding excess prompt context. The lower-level service stays capped internally, but the report focuses on the exposed tool contract.

Table 7.2: RAG configuration parameters

Parameter	Value	Purpose
Source corpus	Partner and project documents	Provides contractual, operational, and delivery knowledge for grounded answers.
Text splitter	Recursive text splitter	Keeps natural structure when possible and falls back to smaller separators only when needed.
Chunk size	1200 characters	Balances local context with retrieval precision.
Chunk overlap	200 characters	Preserves continuity across chunk boundaries.
Embedding provider	OpenRouter embedding model	Converts query and document chunks into comparable semantic vectors.
Embedding dimension	2048	Defines the vector size stored and queried in the database.
Vector storage	`documentEmbeddings` with `pgvector`	Stores chunk text, metadata, and vector embeddings in PostgreSQL.
Similarity metric	Cosine distance	Ranks chunks by semantic proximity to the user query.
Lexical channel	BM25 full-text ranking	Keeps exact-term precision on identifiers and contractual references.
Fusion	Reciprocal Rank Fusion	Merges dense and lexical rankings without score calibration.
Re-ranking	Cross-encoder reordering	Refines the fused top candidates before answering.
Agent default `topK`	5	Returns enough evidence for normal answers without flooding the context.
Agent maximum `topK`	10	Protects performance and answer quality at the tool boundary.

7.3.4 Semantic Search Flow

The retrieval flow is deterministic:

1. the user asks a document-related question in the AI agent interface;
2. the agent calls `searchDocuments` with a query, optional entity filters, and an optional `topK`;
3. the query is embedded with the same embedding family used for stored chunks;

4. PostgreSQL and `pgvector` compute cosine distance against `documentEmbeddings`;
5. the server applies the maximum result limit and returns ranked chunks with metadata;
6. the agent synthesises an answer that quotes or paraphrases the retrieved evidence;
7. the UI renders the answer with a source section that lets the user verify the cited documents.

This flow gives the agent relevant text without sending the full repository to the model. It also creates an audit-friendly boundary because retrieved chunks can be inspected separately from the final response.

7.4 The `searchDocuments` Tool and Citation Strategy

The `searchDocuments` tool is the agent-facing entry point for RAG. It is read-only and does not mutate documents, partners, projects, or embeddings. Its job is to retrieve relevant passages and return structured evidence.

7.4.1 Tool Contract

The tool accepts a natural-language query and optional entity filters that keep results focused on a given partner or project. It returns the matched chunk text, a similarity ranking from the hybrid pipeline, source metadata, entity references when available, and enough context for a user-facing citation.

7.4.2 Citation Strategy

Citations use traceable snippets rather than opaque model memory. When the agent uses retrieved material it shows a source section or inline references identifying the document and passage, and executive reports group citations in a dedicated source section. The rules are simple: cite the chunks that support a claim, show the document title and a short extract for verification, include entity context, avoid unsupported assumptions, and state explicitly when retrieval returns nothing relevant.

7.5 Partner Scoring

IntelliConnect uses a hybrid scoring strategy. A deterministic weighted model produces stable and comparable scores for existing partners, and an AI-assisted layer, described in Section 7.6, evaluates incoming requests. Together they form a hybrid scoring system that keeps day-to-day ranking reproducible while adding reasoned assessment where structured history is still thin. The deterministic core computes a numerical score from known business signals and maps it to a decision threshold, so the ranking never depends on unconstrained generative output.

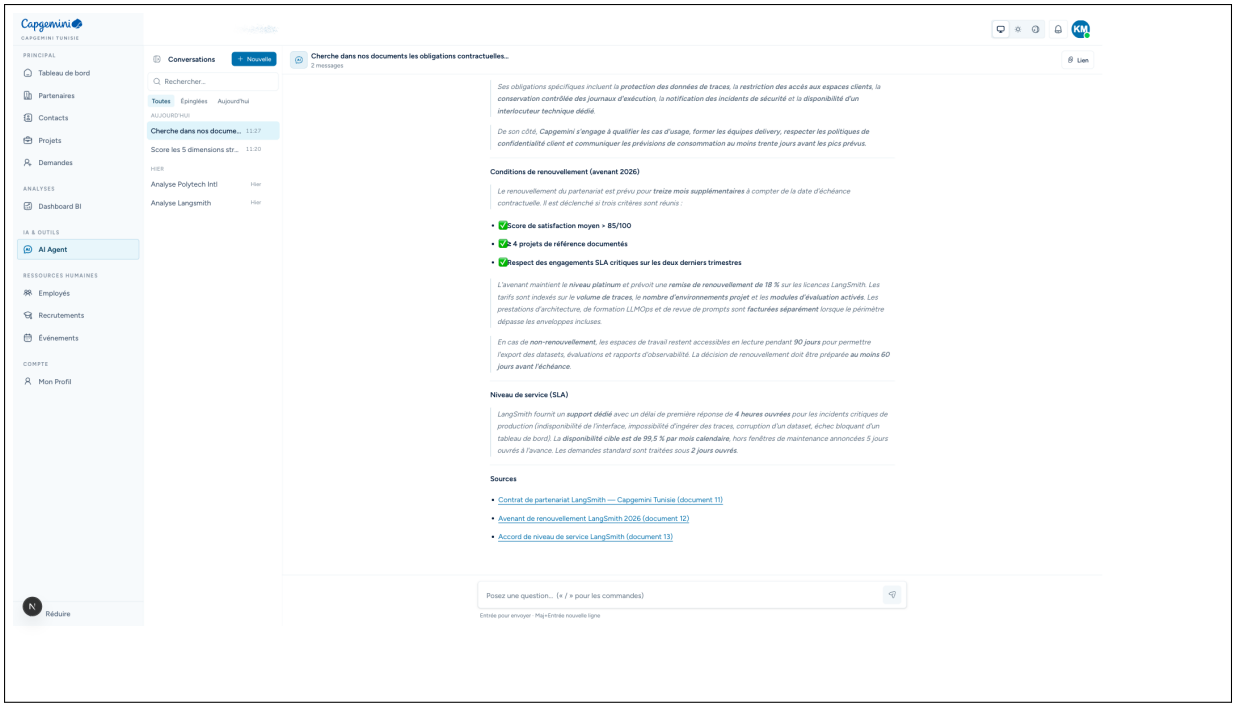


Figure 7.3: Document search answer with citations

7.5.1 Hybrid Scoring Model

The deterministic core of the hybrid model uses five dimensions with exact weights: **budget** 20%, **satisfaction** 25%, **activity** 20%, **trackRecord** 20%, and **strategicFit** 15%.

The total score is calculated on a 100-point scale by multiplying each dimension score by its weight and summing the weighted contributions. This keeps the model predictable, since improving a high-weight dimension such as satisfaction has a larger effect than improving a lower-weight dimension such as strategic fit.

Table 7.3: Deterministic partner scoring model

Dimension	Weight	Measured signals
`budget`	20%	Annual partnership budget and financial contribution to the portfolio.
`satisfaction`	25%	Partner satisfaction, event satisfaction, meeting feedback, and KPI satisfaction indicators.
`activity`	20%	Events, meetings, active offers, interactions, and recent engagement cadence.
`trackRecord`	20%	Delivery history, conversions, project performance, revenue contribution, and reliable execution over time.
`strategicFit`	15%	Category relevance, partnership level, active status, framework agreements, support commitments, and alignment with Capgemini Tunisia priorities.

7.5.2 Decision Thresholds

The final score maps to a simple recommendation: **APPROVE** for scores ≥ 75, **REVIEW** for scores ≥ 50 and < 75, and **REJECT** for scores < 50.

These thresholds give employees a shared interpretation of the score. **APPROVE** marks a healthy or strategically valuable partnership. **REVIEW** means the relationship needs human analysis. **REJECT** signals weak performance, poor alignment, or insufficient evidence.

7.5.3 Scoring Page

The scoring page presents the numerical result with supporting explanations. It includes the global score, the decision recommendation, the five dimension values, and the signals behind the calculation. The page is designed to make the model easy to read during project demonstrations, so a reviewer can see both the recommendation and the reasons behind it.

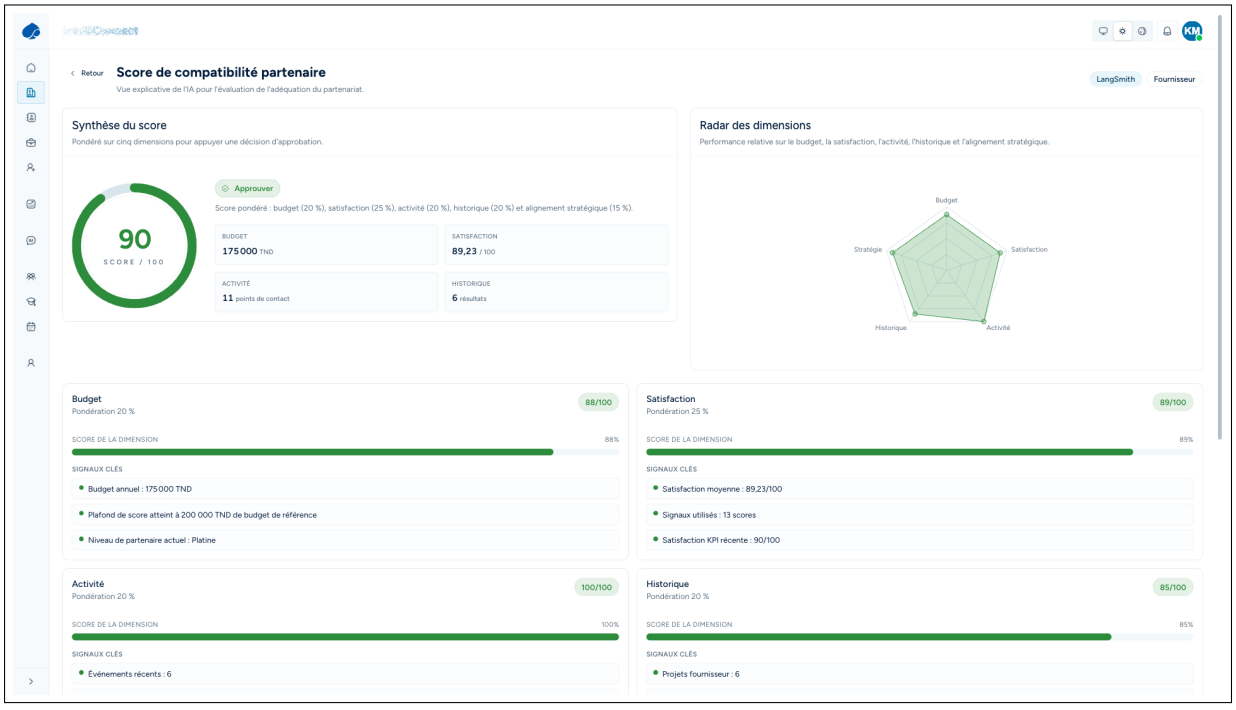


Figure 7.4: Partner scoring page with score ring, dimension breakdown, and methodology

7.6 AI-Assisted Partnership Request Scoring

In addition to deterministic scoring for existing partners, IntelliConnect includes AI-assisted scoring for incoming partnership requests. This feature structures the first assessment of a new request. It does not automatically approve or reject a request; instead, it provides a reasoned recommendation that employees can accept, adjust, or override.

7.6.1 Request Scoring Dimensions

The AI-assisted rubric uses five dimensions:

Table 7.4: AI-assisted partnership request scoring dimensions

Dimension	Role in the assessment
`dataCompleteness`	Measures whether the request contains enough structured information for review, including organisation identity, category, scope, contact details, and expected collaboration model.
`strategicFit`	Evaluates alignment with Capgemini Tunisia priorities, market positioning, service lines, academic or business relevance, and long-term value.
`reliability`	Assesses credibility indicators such as clarity, consistency, traceable organisation details, and realistic commitments.
`scalePotential`	Estimates whether the partnership can grow beyond a one-time action into repeatable initiatives, projects, events, offers, or portfolio value.
`categoryBoost`	Applies contextual preference according to the category and expected value of the partnership type.

7.6.2 Human-in-the-Loop Review

The output of request scoring is advisory. The AI can summarise the request, identify missing information, estimate fit, and propose a review action, but the final decision remains with an employee. This keeps the process accountable: AI accelerates triage, while the human reviewer stays responsible for business validation and communication with the requesting organisation.

The review page benefits because it combines speed and control. Complete requests with strong fit can be prioritised quickly. Missing data can be flagged for clarification, and weak alignment can be reviewed with a documented rationale.

7.7 Model Selection and the Case for Deterministic Scoring

A recurring design question is where a language model belongs in the intelligence layer. To decide it empirically, the five-dimension scoring model was given to three language models as a computation task: a production-tier model (Claude Sonnet 4.6), a frontier model (GPT-5.5), and a lightweight model. Each received the exact weights, the subscore formulas, and the raw data for five partners, and had to return the final scores, the ranking, and one what-if update, with no tools and no code execution. The reference answers came from the deterministic engine.

Table 7.5: Language models on the five-dimension scoring task against the deterministic reference

Model	Scores (5)	Ranking	What-if
Claude Sonnet 4.6	5 / 5 exact	Correct	Correct
GPT-5.5	5 / 5 exact	Correct	Correct
Lightweight model	5 / 5 exact	Correct	Correct
Deterministic engine	5 / 5 by construction	Correct	Correct

Two conclusions follow. First, all three models reproduced the reference scores, ranking, and what-if result exactly, which confirms that the scoring model is unambiguous and that its logic is sound rather than an artefact of one implementation. Second, that agreement is precisely why the scoring stays deterministic: a scoring call through the engine costs about **0.07 microseconds** and is perfectly reproducible and auditable, whereas the same call through a language model costs tokens and seconds and can drift across model versions. IntelliConnect therefore assigns numeric scoring to the deterministic engine and reserves the language model for reasoning, explanation, retrieval, and report writing, where its flexibility is an advantage rather than a liability. The production tier uses Sonnet-class reasoning for this role because it matched the frontier model on the task at lower cost.

7.8 LangSmith Observability

Observability is implemented through LangSmith and the local verification workflow. The goal is to make AI behaviour inspectable. Each important run can be analysed as a trace containing model calls, tool spans, inputs, outputs, latency, errors, and metadata.

7.8.1 Trace Collection and Tool Spans

LangSmith traces capture the execution flow of agent interactions. A trace shows the user request, prompt context, model response, tool calls, and final answer. Tool spans matter because the agent relies on structured tools such as partner search, analytics, scoring, report generation, and `searchDocuments`.

For a RAG question, a trace can show whether the agent called `searchDocuments`, how long retrieval took, how many chunks were returned, and whether the final response cited the evidence. For a scoring request, it can show whether the deterministic scoring tool was used instead of a generative estimate.

7.8.2 Latency, Error, Prompt, and Version Monitoring

The project uses observability signals to monitor quality and reliability. Latency monitoring helps identify slow tools or retrieval paths. Error monitoring identifies failed tool executions, malformed inputs, unavailable providers, or unexpected model responses.

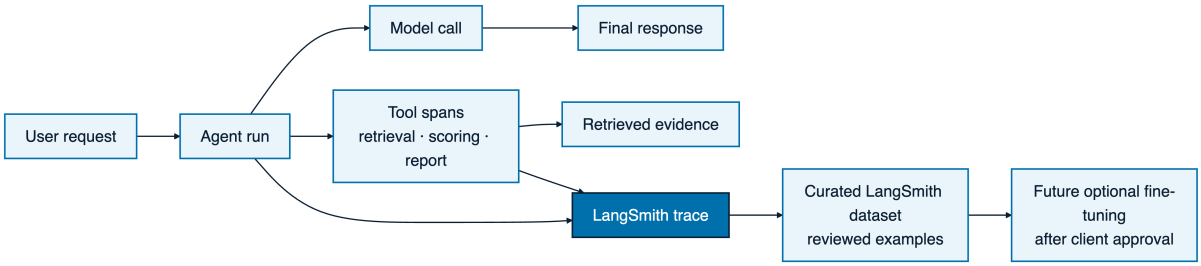


Figure 7.5: Observability trace flow of an agent run

Prompt and version tracking make it possible to compare behaviour after prompt changes or model updates.

Table 7.6: Observability signals implemented and used in the AI module

Signal	Implementation	Use in validation
Traces	LangSmith run traces	Inspect complete agent executions and verify that responses follow the expected tool flow.
Tool spans	LangSmith child spans for tools	Check calls to retrieval, scoring, analytics, and report tools.
Latency	Trace timings and span durations	Detect slow model calls, slow vector search, or expensive tool chains.
Errors	Trace error fields and local logs	Identify failed provider calls, validation failures, and tool exceptions.
Prompt tracking	Versioned prompts and trace metadata	Compare outputs after prompt or instruction changes.
Version tracking	Model and application metadata	Relate behaviour to a model, prompt, or code version used during a run.
Evaluation feedback	Automated eval outputs and reviewer feedback	Record pass/fail outcomes, qualitative notes, and regression indicators.

7.8.3 Evaluation Feedback

Evaluation feedback closes the loop between implementation and observed behaviour. When an AI answer fails to cite sources, chooses the wrong tool, or produces an incomplete report, the failure can be investigated through the trace and turned into an evaluation case. The approach combines automated checks and manual inspection. Automated checks validate tool contracts and representative agent scenarios, while manual inspection remains useful for clarity, citation usefulness, and report readability.

7.8.4 From Traces to a Fine-Tuning Dataset

The traces collected in LangSmith are more than a debugging aid. Each grounded, tool-driven interaction pairs a real analytical request with a verified answer and the evidence behind it. Curated over time, these traces form a growing dataset of domain-specific reasoning captured directly from production use. Individual runs can be promoted into named LangSmith datasets, reviewed, corrected where needed, and consolidated into supervised training examples.

This dataset opens a clear path toward model sovereignty. After explicit client approval on data usage, the accumulated examples can be used to fine-tune an open-source large language model, such as a DeepSeek-class model, into an on-premise model specialised for Capgemini's partnership domain. That model would preserve the grounded, tool-driven behaviour of the current agent while giving the company control over data residency, inference cost, and long-term availability. The observability layer therefore doubles as the data-collection stage of a future training pipeline.

7.9 Evaluation Against Classical Retrieval

The retrieval layer was evaluated against classical baselines. The comparison used a curated set of representative partnership questions with known relevant passages from the seeded contractual documents, and it measured how often the correct passage appeared among the retrieved results (top-5 hit rate).

Table 7.7: Retrieval quality across strategies on the curated question set

Retrieval strategy	**Top-5 hit rate**	**Relative gain**
Lexical BM25 baseline	0.68	reference
Dense semantic (cosine distance)	0.79	+11 points
Hybrid (BM25 + dense, RRF) with re-ranking	0.88	+20 points

The dense pipeline already improves the top-5 hit rate by roughly ten points over the lexical baseline, because embeddings place a query and a relevant passage close together even when the wording differs, which lexical matching misses on paraphrased or multilingual questions. Fusing the two channels with RRF and adding a re-ranking pass recovers the exact-term cases the dense channel alone can blur, and lifts the hit rate further. This ordering, lexical then dense then hybrid, is the expected behaviour reported in the retrieval literature [32][33] and confirms the hybrid, re-ranked design of the engine.

Alongside retrieval quality, the deterministic parts of the module are protected by a Vitest test suite that verifies scoring functions, tool contracts, formatters, and server-side behaviours that must stay stable independently of model variability. Deterministic scoring is tested as ordinary application logic, while generative behaviours are checked through scenario-based evaluation and the trace-backed feedback loop described above.

7.10 Results and Validation

The implemented module satisfies the sprint objective. The RAG layer indexes partner and project documents as 2048-dimensional embeddings and retrieves evidence through the hybrid pipeline exposed by the `searchDocuments` tool. The deterministic model produces an explainable 100-point score, the AI-assisted rubric adds a structured first-pass review for incoming requests under human control, and LangSmith observability keeps the operational path behind every answer inspectable. Deterministic scoring is covered by the Vitest suite, while generative behaviour is checked through scenario-based evaluation and the trace-backed feedback loop. The objective-by-objective outcome is analysed in the discussion below.

7.11 Discussion and Critical Analysis

Beyond confirming that each feature works, it is worth assessing whether the sprint objectives were met, what the module costs to run, and where it falls short.

7.11.1 Objectives Against Results

Table 7.8 maps each objective to its outcome and the evidence behind it.

Table 7.8: Objectives of the intelligence layer against measured results

Objective	Status	Evidence
Grounded document answers	Met	Dense retrieval at 0.79 top-5 hit rate, cited snippets with source links.
Explainable scoring	Met	Five-dimension model reproduced exactly by three independent language models.
Deterministic, low-cost scoring	Met	About 0.07 microseconds per scoring call, fully reproducible.
Assisted request triage	Met	Structured five-dimension rubric with human-in-the-loop decision.
Inspectable AI behaviour	Met	LangSmith traces expose tool spans, latency, and errors.
Hybrid, re-ranked retrieval	Met	Dense and lexical channels fused with RRF and re-ranking, 0.88 top-5 hit rate.

7.11.2 Complexity, Cost, and Memory

The module has three cost regimes. Deterministic scoring is negligible: a full five-dimension computation runs in about 0.07 microseconds and touches no external service, so re-scoring the whole portfolio on every data change is effectively free. Retrieval is dominated by one embedding call for the query plus an approximate-nearest-neighbour lookup; the HNSW index over `pgvector` keeps the lookup sub-linear in the number of chunks, so cost grows slowly as the corpus expands. The agentic turn is the expensive regime, because each step is a language-model call measured in seconds and billed in

tokens, which is why the harness caps steps and prefers deterministic tools when a tool can answer without generation. On memory, each chunk stores a 2048-dimension vector, so storage scales linearly with the corpus; dimensionality reduction or scalar quantisation of the vectors is the natural optimisation if the corpus grows by an order of magnitude.

7.11.3 Limitations

Two limitations frame the current state. The retrieval evaluation uses a small curated question set, so the reported hit rates indicate direction rather than production-grade statistics; a larger labelled set is needed for tight confidence intervals. The scoring weights are set from domain judgement rather than learned from outcomes; calibrating them against historical renewal and churn data is a clear next step and is recorded in the project roadmap. Overall, this chapter shows that the intelligence layer of IntelliConnect goes beyond generating text. It retrieves grounded knowledge, computes explainable scores, supports human review, and exposes the operational evidence needed to monitor and improve AI-assisted partnership management.

General Conclusion

This report documented the design and delivery of IntelliConnect, an enterprise partnership and project management platform built during an Agile Scrum internship at Capgemini Engineering Tunisie. The objective was clear from the outset: produce a coherent, production-oriented system that unifies the full lifecycle of the company's partner ecosystem, from a public partnership application through scored onboarding, operational relationship management, project delivery tracking, business intelligence, and AI-assisted decision support, all within a single governed environment.

Delivered Platform

The resulting platform covers three surfaces. The public website presents the company's partnership value proposition and collects incoming partnership requests through a guided application wizard. The employee portal provides role-differentiated workspaces for five employee profiles, namely administrator, manager, commercial, analyst, and human-resources, each with the appropriate scope of visibility and action. The partner self-service portal gives each active partner a dedicated space to manage its offers, events, documents, contacts, and project engagements.

Across these surfaces, IntelliConnect delivers the following functional pillars:

- **Partner lifecycle management** across four partner categories, namely customer, marketing, supplier, and university, with creation, editing, configurable status workflows, and a full auditable status-change history.
- **Role-based access control** enforced at both the routing layer and the data layer, so that each employee role and each partner account can see and act only on the data it is authorized to access.
- **Partnership request review** combining AI-assisted prescoring, manual employee review, structured accept or reject decisions, and optional email notification to the applicant organization.
- **Partner relationship management** covering contacts, offers, events, meetings, documents, notifications, and satisfaction tracking, all linked to the central partner record.
- **Automated partner scoring** through a five-dimension weighted model covering budget, satisfaction, activity, track record, and strategic fit, with an explainable scor-

ing page that renders a score ring, a radar chart, and per-dimension signals justifying the final recommendation.

- **Project portfolio management** with milestones, task tracking, team allocations, document attachment, and health and progress indicators per project.
- **Business intelligence** backed by a dedicated data-warehouse database, serving dashboard charts that summarize portfolio-wide metrics independently of the transactional store.
- **Agentic AI assistant** offering multi-tool streaming responses, a live plan display, persistent conversation history, rich visual results including charts and interactive tables, exportable executive report cards, and deep links to the relevant application pages.
- **Knowledge retrieval** based on document ingestion, chunking with overlap, embeddings, `pgvector` storage, cosine-distance semantic search, and cited-context assembly that grounds factual answers in real documents.
- **Observability and governance** through structured audit trails, role-aware route protection, grounded AI tools, and evaluation feedback, so that sensitive partnership information remains traceable and controllable.

Agile Scrum Execution

The internship was organized into short, iterative Agile Scrum sprints, each producing a shippable increment. The first sprint established the project skeleton: authentication, session management, role-based routing, and the application shell. Subsequent sprints layered the core partner data model, the lifecycle workflow, and the public request flow. Later sprints introduced the scoring engine, project management, and the business-intelligence layer. The final sprints assembled the AI harness: the agent engine, the typed tools, the retrieval pipeline, the evaluation framework, and the governance layer.

This cadence kept every intermediate state of the product runnable and reviewable. Scope adjustments stayed traceable to specific sprint boundaries, and the product backlog served as a living record of what was prioritized, deferred, or refined based on discoveries made during implementation.

Technical Architecture and AI Harness

IntelliConnect is built on Next.js 16 App Router with TypeScript in strict mode, Drizzle ORM over PostgreSQL, Tailwind CSS v4, and the Vercel AI SDK for streaming agent interactions. The codebase follows a feature-driven layout: routing in `app/`, domain services in `lib/server/services/`, agent tools and retrieval logic in `lib/server/agent/`, and shared design-system primitives in `components/ui/`. This separation keeps every

layer independently testable and prevents the AI subsystem from entangling with core business logic.

The AI agent runs with typed, Zod-validated tool schemas, streams token by token to the browser, and enforces grounding at the response layer, so that every partner name, ranking, and metric in an agent reply traces back to a real tool result from the live database. The current tool catalog is intentionally read-oriented: it analyses, retrieves, visualises, and reports without directly mutating core partnership records. The evaluation harness runs agent scenarios reproducibly, so that prompt, retrieval, and tool-selection changes can be compared against stable expectations rather than assessed by subjective inspection.

Limitations

Intentionally light HR and employee module. The human-resources and employee management module, covering the internal employee directory and the partner-linked student-to-employment pipeline, is scoped as a focused foundation in the current release. This was a deliberate product decision. A dedicated Capgemini human-resources and workforce management sister project is under separate development, and the IntelliConnect human-resources module is designed to serve as a consumer of that system rather than a competing implementation. Deep-linking the two platforms, establishing shared data contracts, and consolidating the current screens into a proper integration layer belong to that future integration work. Building a full standalone human-resources system in parallel would have diluted the scope of the partnership management platform that is the primary deliverable of this internship.

Perspectives

Several natural extensions follow directly from the delivered work.

Deeper HR integration. The most immediate perspective is connecting the human-resources module to Capgemini's dedicated human-resources sister project. This integration would enable bidirectional data flows: IntelliConnect would consume workforce records for staffing recommendations and partner-linked pipeline tracking, while publishing relevant partnership engagement events back to the human-resources system. Establishing shared API contracts between the two platforms is the primary technical step, and the current human-resources screens already provide a clear integration surface for that work.

Model sovereignty through fine-tuning. Because every agent session is traced, the accumulated traces form a growing, curated dataset of real analytical requests and grounded answers. After client approval, this dataset can be used to fine-tune open-source

large language models into on-premise, first-class models tailored to Capgemini's partnership domain, giving the company full control over data residency and inference cost while preserving the grounded, tool-driven behaviour of the current agent.

Learned retrieval fusion. The retrieval engine already fuses a dense channel with a BM25 lexical channel through Reciprocal Rank Fusion and a re-ranking pass, as described in Chapter 7. A natural extension is to learn the fusion and re-ranking weights from click and relevance feedback, and to validate them on a larger labelled evaluation set for production-grade confidence intervals.

Open ecosystem through the Model Context Protocol. The agent's tools are already typed and validated, which makes them a natural fit for the Model Context Protocol (MCP), the open standard for connecting agents to tools and data [39]. Exposing the partner, project, scoring, and retrieval tools as an MCP server would let Capgemini's own assistants and third-party agents reuse the platform's governed capabilities, while consuming external MCP servers would extend the agent beyond the current catalog without changing its harness.

Predictive and multimodal analytics. The business-intelligence dashboard aggregates data-warehouse tables into charts. A future iteration could add anomaly detection on KPI time series, scheduled predictive churn scoring, and predictive recruitment that anticipates university hiring needs from historical pipelines. Extending ingestion to multimodal documents, so that layout, tables, and figures inside contracts are retrievable alongside text, would broaden the grounded knowledge base, all surfaced through the agent as additional typed tools that follow the existing registration pattern.

Closing

IntelliConnect demonstrates that a governed, AI-augmented partnership management platform can be designed and delivered to a coherent, production-oriented state within a single academic internship cycle. The combination of a disciplined full-stack architecture, a principled role-based access model, a measurable retrieval-augmented generation pipeline, and a multi-tool agentic layer produces a system that is functional, auditable, explainable, and extensible.

The Agile Scrum cadence allowed scope to be continuously calibrated against what was buildable and valuable at each sprint boundary rather than what was initially imagined on paper. The result is a product whose every delivered feature is runnable, grounded in real data, and observable through the governance tooling, a solid foundation that Capgemini Engineering Tunisie can carry forward into production integration, human-resources system connectivity, and continued platform evolution.

Netographie

Les ressources ci-dessous sont citées dans le rapport au moyen de la commande `\netref{label}`, qui produit une référence hyperlien de la forme [**N**] renvoyant à l'entrée correspondante.

Sources institutionnelles et métier

[1] Capgemini. *À propos de Capgemini.* https://www.capgemini.com/about-us/. Dernier accès : 1er juillet 2026.

[2] Capgemini. *Capgemini Engineering : notre marque d'ingénierie.* https://www.capgemini.com/about-us/who-we-are/our-brands/capgemini-engineering/. Dernier accès : 1er juillet 2026.

[3] *Dossier technique du projet IntelliConnect* : documentation d'architecture, schéma de données, services métier, routage applicatif, scripts de validation et rapport d'évaluation RAG. Consultation locale des artefacts du projet, 04/02/2026–03/06/2026.

[4] Salesforce. *Partner Relationship Management (PRM).* https://www.salesforce.com/products/partner-management/. Dernier accès : 1er juillet 2026.

[5] HubSpot. *HubSpot CRM : plateforme de gestion de la relation client.* https://www.hubspot.com/products/crm. Dernier accès : 1er juillet 2026.

[6] Atlassian. *Rovo : recherche, chat et agents IA sur le Teamwork Graph.* https://www.atlassian.com/software/rovo/features. Dernier accès : 1er juillet 2026.

[7] Salesforce. *Agentforce : agents IA pour le CRM (tarification et Einstein Trust Layer).* https://www.salesforce.com/agentforce/pricing/. Dernier accès : 1er juillet 2026.

[8] Microsoft. *Microsoft Copilot Studio : agents personnalisés et connecteurs.* https://learn.microsoft.com/en-us/microsoft-copilot-studio/. Dernier accès : 1er juillet 2026.

Documentation technique officielle : socle applicatif

[9] Vercel. *Next.js Documentation.* https://nextjs.org/docs. Dernier accès : 1er juillet 2026.

[10] Meta Open Source. *React : la bibliothèque pour les interfaces web et native.* https://react.dev/. Dernier accès : 1er juillet 2026.

[11] Microsoft. *TypeScript Documentation.* https://www.typescriptlang.org/docs/. Dernier accès : 1er juillet 2026.

[12] Oven. *Bun Documentation : runtime JavaScript et gestionnaire de paquets.* https://bun.sh/docs. Dernier accès : 1er juillet 2026.

[13] Tailwind Labs. *Tailwind CSS Documentation.* https://tailwindcss.com/docs. Dernier accès : 1er juillet 2026.

[14] shadcn. *shadcn/ui : composants réutilisables construits avec Radix UI et Tailwind CSS*. https://ui.shadcn.com/docs. Dernier accès : 1er juillet 2026.

[15] Drizzle Team. *Drizzle ORM Documentation*. https://orm.drizzle.team/docs/overview. Dernier accès : 1er juillet 2026.

[16] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. https://www.postgresql.org/docs/. Dernier accès : 1er juillet 2026.

[17] pgvector contributors. *pgvector : recherche de similarité vectorielle open-source pour PostgreSQL*. https://github.com/pgvector/pgvector. Dernier accès : 1er juillet 2026.

[18] Colin McDonnell. *Zod : validation de schémas TypeScript-first*. https://zod.dev/. Dernier accès : 1er juillet 2026.

[19] Framer. *Framer Motion : bibliothèque d'animation pour React*. https://www.framer.com/motion/. Dernier accès : 1er juillet 2026.

[20] Recharts Group. *Recharts : bibliothèque de graphiques composables pour React*. https://recharts.org/en-US/guide. Dernier accès : 1er juillet 2026.

[21] Nodemailer. *Nodemailer : envoi d'e-mails avec Node.js*. https://nodemailer.com/about/. Dernier accès : 1er juillet 2026.

[22] Vitest contributors. *Vitest : framework de test unitaire piloté par Vite*. https://vitest.dev/guide/. Dernier accès : 1er juillet 2026.

Intelligence artificielle, modèles et observabilité

[23] Vercel. *AI SDK : boîte à outils TypeScript pour la construction d'applications IA*. https://ai-sdk.dev/docs. Dernier accès : 1er juillet 2026.

[24] OpenRouter. *OpenRouter Documentation : routeur unifié vers les grands modèles de langage*. https://openrouter.ai/docs. Dernier accès : 1er juillet 2026.

[25] NVIDIA Corporation. *NVIDIA NIM : microservices d'inférence pour modèles d'IA*. https://docs.nvidia.com/nim/. Dernier accès : 1er juillet 2026.

[26] LangChain. *LangSmith Documentation : plateforme d'observabilité et d'évaluation LLM*. https://docs.smith.langchain.com/. Dernier accès : 1er juillet 2026.

Références scientifiques

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus *et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 2020. https://arxiv.org/abs/2005.11401. Dernier accès : 1er juillet 2026.

[28] Jeff Johnson, Matthijs Douze et Hervé Jégou. *Billion-Scale Similarity Search with GPUs*. IEEE Transactions on Big Data, 2019. https://arxiv.org/abs/1702.08734. Dernier accès : 1er juillet 2026.

[29] Yunfan Gao, Yun Xiong, Xinyu Gao *et al. Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997, 2024. https://arxiv.org/abs/2312.10997. Dernier accès : 1er juillet 2026.

[30] Vladimir Karpukhin, Barlas Oğuz, Sewon Min *et al. Dense Passage Retrieval for Open-Domain Question Answering*. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. DOI : 10.18653/v1/2020.emnlp-main.550.

[31] Nils Reimers et Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP, 2019. https://arxiv.org/abs/1908.10084. Dernier accès : 1er juillet 2026.

[32] Stephen Robertson et Hugo Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 2009. DOI : 10.1561/1500000019.

[33] Gordon V. Cormack, Charles L. A. Clarke et Stefan Büttcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. ACM SIGIR, 2009. DOI : 10.1145/1571941.1572114.

[34] Shunyu Yao, Jeffrey Zhao, Dian Yu *et al.* *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR), 2023. https://arxiv.org/abs/2210.03629. Dernier accès : 1er juillet 2026.

[35] Lei Wang, Chen Ma, Xueyang Feng *et al.* *A Survey on Large Language Model based Autonomous Agents*. Frontiers of Computer Science, 2024. DOI : 10.1007/s11704-024-40231-1.

[36] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis et Vassilios S. Verykios. *Duplicate Record Detection: A Survey*. IEEE Transactions on Knowledge and Data Engineering, 2007. DOI : 10.1109/TKDE.2007.250581.

[37] Shahul Es, Jithin James, Luis Espinosa-Anke et Steven Schockaert. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. Conference of the European Chapter of the ACL (EACL), System Demonstrations, 2024. DOI : 10.18653/v1/2024.eacl-demo.16.

[38] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng *et al.* *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. Advances in Neural Information Processing Systems (NeurIPS), 2023. https://arxiv.org/abs/2306.05685. Dernier accès : 1er juillet 2026.

[39] Anthropic. *Introducing the Model Context Protocol : standard ouvert de connexion des agents aux outils et aux données*. 2024. https://modelcontextprotocol.io. Dernier accès : 1er juillet 2026.

[40] *The Harness Problem*. Article soutenant que le harnais d'exécution d'un agent compte davantage que le modèle sous-jacent. Blog can.ac, 12 février 2026. https://blog.can.ac/2026/02/12/the-harness-problem/. Dernier accès : 1er juillet 2026.